

构成字节流的所有段都有一个 20 个字节长的头。大量的工作针对 TCP 性能进行了优化，它们主要是 Nagle、Clark、Jacobson、Karn 和其他的算法。

网络性能通常由协议和段的处理开销来主宰，在速度很高的时候这种状况变得更糟。协议应该被设计成最小化段的数量，而且在带宽延迟乘积大的路径上工作良好。对于千兆网络，要求使用尽可能简单的协议，并实行流水线式处理。

延迟容忍网络提供了一个跨网络的传递服务，这种网络具有偶尔的连通性或链路的延迟很长。中间节点存储、携带并转发捆绑在一起的数据束，因此最终得以传递到目的地，即使在任何时间都不存在一条从发送端到接收端的工作路径。

习 题

1. 在我们的图 6-2 所示传输原语例子中，LISTEN 是一个阻塞调用。试问这是严格要求的吗？如果不是，请解释如何使用一个非阻塞的原语。与正文中描述的方案相比，你的方案有什么优点？
2. 传输服务原语假设在两个端点之间建立连接的过程是不对称的，一端（服务器）执行 LISTEN，而另一端（客户端）执行 CONNECT。然而，在对等应用中，比如 BitTorrent 那样的文件共享系统，所有的端点都是对等的。没有服务器或客户端功能之分。试问如何使用传输服务原语来构建这样的对等应用？
3. 在图 6-4 所示的底层模型中，它的假设条件是网络层的数据包有可能被丢失，因此，数据包必须被单独确认。假如网络层百分之百可靠，并且永远不会丢失数据包，试问图 6-4 需要做什么修改吗？如果需要的话。
4. 在图 6-6 的两部分中，有一条注释说明了 SERVER_PORT 在客户机和服务器中必须相同。试问为什么这一条如此重要？
5. 在 Internet 文件服务器例子中（图 6-6），除了服务器端的监听队列为满之外，试问还有其他原因能导致客户机的 connect（）系统调用失败吗？假设网络完美无缺。
6. 评判一个服务器是否全程活跃，或者通过进程服务器来按需启动它的一个标准是它所提供的服务的使用频率。试问你能想出作出这一决定的任何其他标准吗？
7. 假设采用时钟驱动方案来生成初始序号，该方案用到了一个 15 位的时钟计数器。每隔 100 毫秒时钟滴答一次，最大数据包生存期为 60 秒。试问，每隔多久需要重新同步一次？
 - (a) 在最差情况下。
 - (b) 当数据每分钟用掉 240 个序号的时候。
8. 试问，为什么最大数据包生存期 T 必须足够大，大到确保不仅数据包本身而且它的确认也消失在网络中？
9. 想象用两次握手过程而不是三次握手过程来建立连接。换句话说，第三个消息不再是要求的。试问现在有可能死锁吗？请给出一个例子说明存在死锁，或者证明死锁不存在。
10. 想象一个广义的 n-军队对垒问题，在这里任何两支蓝军达成一致意见后就足以取得胜利。试问是否存在一个能保证蓝军必赢的协议？
11. 请考虑从主机崩溃中恢复的问题（图 6-18）。如果写操作和发送确认之间的间隔可以设

置得相对非常小，那么，为了使协议失败的概率最小，试问两种最佳的发送端-接收端策略是什么？

12. 在图 6-20 中，假设加入了一个新的流 E，它的路径为从 R1 到 R2，再到 R6。试问对于 5 个流的最大-最小带宽分配有什么变化？
13. 请讨论信用协议与滑动窗口协议的优缺点。
14. 拥塞控制的公平性方面有一些其他的策略，它们是加法递增加法递减（AIAD, Additive Increase Additive Decrease）、乘法递增加法递减（MIAD, Multiplicative Increase Additive Decrease）、乘法递增乘法递减（MIMD, Multiplicative Increase Multiplicative Decrease）。请从收敛性和稳定性两个方面来讨论这三项政策。
15. 试问，为什么会存在 UDP？用户进程使用原始 IP 数据包还不够吗？
16. 请考虑一个建立在 UDP 之上的简单应用层协议，它允许客户从一个远程服务器获取文件，而且该服务器位于一个知名地址上。客户端首先发送一个请求，该请求中包含了文件名；然后服务器以一个数据包序列作为响应，这些数据包中包含了客户所请求的文件的不同部分。为了确保可靠性和顺序递交，客户和服务器使用了停-等式协议。忽略显然存在的性能问题，试问你还能看得出这个协议存在的另一个问题吗？请仔细想一想进程崩溃的可能性。
17. 一个客户通过一条 1 Gbps 的光纤向 100 千米以外的服务器发送一个 128 字节的请求。试问在远过程调用中这条线路的效率是多少？
18. 继续考虑上一个问题的情形。对于给定的 1 Gbps 线路和 1 Mbps 线路两种情况，请计算最小的可能响应时间。试问由此你得出了什么结论？
19. UDP 和 TCP 在传递消息时，都使用了端口号来标识接收方实体。请给出两个理由说明为什么这两个协议要发明一个新的抽象 ID（端口号），而不用进程 ID？在设计这两个协议的时候，进程 ID 早已经存在。
20. 一些 RPC 实现为客户端提供了一个选项，使用实现在 UDP 之上的 RPC 还是使用实现在 TCP 之上的 RPC。试问在什么样的条件下，客户端更喜欢使用基于 UDP 的 RPC？在什么条件下，他或许更喜欢使用基于 TCP 的 RPC？
21. 考虑两个网络 N1 和 N2，在源端 A 和目的端 D 之间有相同的平均延迟。在 N1 中，不同的数据包所经历的延迟是均匀分布，且最大延迟为 10 秒；而在 N2 中，99%的数据包所经历的延迟都小于 1 秒，且没有最大延迟上界。请讨论在这两种情况下如何用 RTP 来传输实时音频/视频流。
22. 试问最小 TCP MTU 的总长度是多少？包括 TCP 和 IP 的开销，但是不包括数据链路层的开销。
23. 数据报的分段和重组机制由 IP 来处理，对于 TCP 不可见。试问，这是否意味着 TCP 不用担心数据错序到达的问题？
24. RTP 被用来传输 CD 品质的音频，这样的音频信号包含一对 16 位的采样值，采样的频率为每秒钟 44 100 次，每个采样值对应于一个立体声声道。试问 RTP 每秒钟必须传输多少个数据包？
25. 试问有可能将 RTP 代码放到操作系统内核中，与 UDP 代码放在一起吗？请解释你的答案。

26. 主机 1 上的一个进程已经被分配了端口 p , 主机 2 上的一个进程也已经被分配了端口 q , 试问这两个端口之间有可能同时存在两个或者多个 TCP 连接吗?
27. 在图 6-36 中, 我们看到除了 32 位的确认号字段外, 在第四个字节还有一个 ACK 标志位。试问这个标志位有额外的意义吗? 为什么有? 或者为什么没有?
28. TCP 段的最大有效载荷为 65 495 字节。试问为什么选择如此奇怪的数值?
29. 描述从图 6-39 中进入 SYN RCVD 状态的两种途径。
30. 请考虑在一条往返时间为 10 毫秒的无拥塞线路上使用慢速启动算法的效果。接收窗口为 24 KB, 最大段长为 2 KB。试问需要多长时间才能首次发送满窗口的数据?
31. 假设 TCP 的拥塞窗口被设置为 18 KB, 并且发生了超时。如果接下来的 4 次突发传输全部成功, 试问拥塞窗口将达到多大? 假设最大段长为 1 KB。
32. 如果 TCP 往返时间 RTT 的当前值是 30 毫秒, 紧接着分别在 26、32、24 毫秒确认到达, 那么, 若使用 Jacobson 算法, 试问新的 RTT 估计值为多少? 请使用 $\alpha = 0.9$ 。
33. 一台 TCP 机器正在通过一条 1 Gbps 的信道发送满窗口的 65 535 字节数据, 该信道的单向延迟为 10 毫秒。试问可以达到的最大吞吐量是多少? 线路的效率是多少?
34. 一台主机在一条线路上发送 1500 字节的 TCP 有效载荷, 最大数据包生存期为 120 秒, 要想不让序号回绕, 试问该线路的最快速度为多少? 要考虑 TCP、IP 和以太网的开销。假设可以连续发送以太网帧。
35. 为了解决 IPv4 的局限性, 主要经过 IETF 的努力, 导致了 IPv6 的设计, 但仍然有很多人不愿意采用这个新版本。然而, 却没有人做出解决 TCP 限制所需要的重大努力。请解释为什么会这样。
36. 在一个网络中, 最大段长为 128 字节, 段的最大生存期为 30 秒, 序号为 8 位, 试问每个连接的最大数据率是多少?
37. 假设你正在测量接收一个段所需要的时间。当发生一个中断时, 你读出系统时钟的值 (以毫秒为单位)。当该段被完全处理后, 你再次读出时钟的值。你测量的结果是 270 000 次为 0 毫秒, 730 000 次为 1 毫秒。试问接收一个段需要多长时间?
38. 一个 CPU 执行指令的速率为 1000 MIPS。数据复制可以按每次 64 位来进行, 每个字的复制需要花费 10 条指令。如果一个入境数据包要被复制 4 次, 试问这个系统能处理一条 1 Gbps 的线路吗? 为了简单起见, 假设所有的指令, 包括读或者写内存的指令, 都以 1000 MIPS 的全速率运行。
39. 为了避开当序号回绕时老的数据包仍然存在这个问题, 可以使用 64 位序号。然而, 从理论上讲, 光纤的运行速度可以达到 75 Tbps。试问在未来的 75 Tbps 网络中使用 64 位序号, 数据包的最大生存期为多少才能确保不会发生回绕问题? 假设每个字节都有自己的序号, 像 TCP 那样。
40. 在 6.6.5 节中, 我们计算了主机以 80 000 包/秒的速度往一条千兆线路发送, 只使用了 6250 指令, 留下一半的 CPU 时间给应用程序。这里计算时假设数据包大小为 1500 字节。针对 ARPANET 大小的数据包 (128 字节) 重复上述计算。在这两种情况下, 假设给出的数据包大小均包括了所有开销。
41. 对于一个运行在 4000 千米距离上的 1 Gbps 网络, 限制因素是延迟而非带宽。请考虑这样一个 MAN, 源端和接收方之间的平均距离为 20 千米。试问多大的速率使得 1 KB

数据包的传输延迟等于光速的往返延迟？

42. 试计算下列网络的带宽-延迟乘积：（1）T1（1.5 Mbps）；（2）以太网（10 Mbps）；（3）T3（45 Mbps）和（4）STS-3（155 Mbps）。假设 RTT 为 100 毫秒。请回忆 TCP 头有 16 位保留用作窗口大小（Window Size）。根据你的计算，试问它有什么隐含的意义吗？
43. 对于地球同步卫星上的一条 50 Mbps 信道，试问它的带宽-延迟乘积是多少？如果所有的数据包都是 1500 字节（包括开销），试问窗口应该为多大（按数据包为单位）？
44. 图 6-6 所示的文件服务器远非完美，它在许多方面都还可以改进。请作以下修改：
 - （a）给客户增加第三个参数，它指定了一个字节范围。
 - （b）给客户增加一个标志 *w*，允许将文件写入服务器上。
45. 所有的网络协议都需要的一种常见功能是处理消息。回想一下，协议处理消息通过添加/拆除头来实现的。有些协议还可能将一个单一消息分解成多个片段，在稍后再把这些多个片段组合成一个单一消息。为此，请设计并实施一个消息管理库。该库提供了创建一个新消息、附加一个消息头到消息上、从消息上剥离消息头、将一个消息分成两个消息、将两个消息组合成一个消息，以及保存消息副本功能。你的实现必须尽量减少把数据从一个缓冲区复制到另一个缓冲区的操作。至关重要的是处理消息的操作不应该触碰到消息的数据，而只处理消息指针。
46. 设计和实现一个聊天系统，它允许多组用户同时进行聊天活动。有一个聊天协调器位于某一个知名的网络地址上，它使用 UDP 与聊天客户通信，并且为每个聊天会话建立聊天服务器，而且还维护了一个聊天会话目录。每个聊天会话都有一个聊天服务器。聊天服务器使用 TCP 与客户端进行通信。聊天客户端允许用户启动、加入或者离开一个聊天会话。请设计和实现聊天协调器、聊天服务器和聊天客户端代码。

第7章 应用层

在完成了所有预备知识的学习后，我们现在来到应用层，这里可以找到所有的网络应用。应用层下面的各层提供了传输服务，但它们并不真正为用户工作。在本章中，我们将学习一些实际的网络应用。

然而，即使在应用层也仍然需要协议的支持，以便各种应用程序能够工作。因此，在开始介绍这些应用之前，我们将先介绍其中的一个协议。这个协议就是 DNS，它负责处理 Internet 命名问题。之后，我们将介绍 3 个实际应用：电子邮件、万维网（World Wide Web）和多媒体。我们将以对内容分发应用的简单介绍来结束本章，包括对等网络的基本概念。

7.1 DNS——域名系统

虽然在理论上，所有程序通过使用它们存储的计算机网络地址（例如 IP），就可以访问 Web 主页、邮箱和其他资源，但是这些地址很难让人记住。而且浏览一个公司在 128.111.24.41 上的 Web 主页，意味着如果该公司将主页移到了另一台机器上，且该机器具有了不同的 IP 地址时，那么必须将该机器的 IP 地址通知到每个人。因此，人们引入了可读性好的高层名字，以便将机器名字与机器地址分离开。在这种方式下，无论真正实用的 IP 地址是什么，人们熟知的公司 Web 服务器为 www.cs.washington.edu。然而，因为网络本身只能理解数字形式的地址，所以需要某种机制将名字转换成网络地址。下面，我们学习如何在 Internet 上完成这种从名字到地址的映射。

我们回到早期的 ARPANET，那时只有一个简单的文件，名叫 `hosts.txt`，它列出了所有的计算机名字和它们的 IP 地址。每天晚上，所有主机都从一个维护此文件的站点将该文件取回，然后在本地进行更新。对于一个拥有几百台大型分时机器的网络而言，这种方法工作得相当好。

然而，当几百万台 PC 连接到 Internet 以后，使用网络的每个人都意识到这种方法将不能继续有效工作了。一方面，这个文件变得非常庞大。然而，更重要的是，除非集中管理机器名字，否则主机名冲突的现象将会频繁发生；而且在一个巨大的国际性网络中，由于负载和延迟，要实现这种集中式的管理简直难以想象。为了解决这些问题，1983 年人们发明了域名系统（DNS，Domain Name System）。自发明以来，它一直是 Internet 的关键组成部分。

DNS 的本质是发明了一种层次的、基于域的命名方案，并且用一个分布式数据库系统加以实现。DNS 的主要用途是将主机名映射成 IP 地址，但它也可以用于其他用途。RFC 1034、1035、2181 给出了 DNS 的定义，后来其他文档对它又做了进一步的阐述。

简要地说，DNS 的使用方法如下所述。为了将一个名字映射成 IP 地址，应用程序调用一个名为解析器（resolver）的库程序，并将名字作为参数传递给此程序。在图 6-6 中我

们看到的 `gethostbyname` 就是一个解析器例子。解析器向本地 DNS 服务器发送一个包含该名字的请求报文；本地 DNS 服务器查询该名字，并且返回一个包含该名字对应 IP 地址的响应报文给解析器，然后解析器再将 IP 地址返回给调用方。查询报文和响应报文都作为 UDP 数据包发送。有了 IP 地址以后，应用程序就可以与目标主机建立一个 TCP 连接，或者给它发送 UDP 数据包。

7.1.1 DNS 名字空间

管理一个大型并且经常变化的名字集合不是个简单的问题。在邮政系统中，名字管理要求所有的信件必须（隐式地或显式地）指定国家、州或省、城市、街道地址以及收件人的名字。通过这种地址的层次结构，纽约州 White Plains 市 Main 大街上的 Marvin Anderson 与德克萨斯州奥斯汀市 Main 大街上的 Marvin Anderson 就不会产生混淆。DNS 采用了同样的工作方式。

对于 Internet，命名层次结构的顶级由一个专门组织负责管理。该组织名为 Internet 名字与数字地址分配机构（ICANN，Internet Corporation for Assigned Names and Numbers），创建于 1998 年，它是 Internet 成长为全球性并且受到经济关注的部分成熟标志。从概念上讲，Internet 被划分为超过 250 个顶级域名（top-level domains），其中每个域涵盖了许多主机。这些域又被进一步划分成子域，这些子域可被再次划分，依此类推。所有这些域可以表示为一棵树，如图 7-1 所示。树的叶子代表没有子域的域（当然要包含机器）。一个叶结点域可能包含一台主机，或者代表一个公司，该公司包含数以千计的主机。

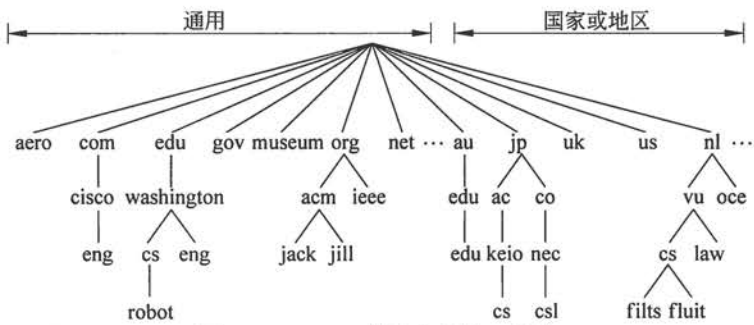


图 7-1 Internet 域名空间的一部分

顶级域名分为两种类型：通用的和国家或地区的。在图中 7-2 中列出的通用域名包括从 20 世纪 80 年代开始的原始域名和向 ICANN 申请引入的新域名。将来或许会增加其他的通用顶级域名。

国家或地地区域名包括每个国家或地区，国家或地区的域名由 ISO 3166 文档定义。2010 年推出了使用非拉丁字母的国际化国家或地地区域名。这些域名使得说阿拉伯语、西里尔文、中文或其他语言国家的人以自己的母语来命名他们的主机。

获得一个二级域名容易得多，比如 `name-of-company.com`。顶级域名由 ICANN 委任的注册机构（registrar）负责运行。要获得一个域名，只要到相应的注册机构（在这种情况下是 `com`），检查所需的名称是否可用，并且不是别人的商标。如果没有问题，请求注册域名的一方向管理域名的注册方支付一小笔年费，即可得到心仪的域名。

然而，随着 Internet 变得越来越商业化和越来越国际化，它也变得更具争议性，特别是在有关域名的事务上。这样的争议包括 ICANN 本身。例如，×××域名创造了好几年，法院已经审理要解决它。自愿将成人内容放在自己的域名空间中到底是好事还是坏事（有些人不希望成人内容对 Internet 上的所有人都开放，而其他则想把成人内容放在一个域名中，以便让保姆过滤器很容易地找到并阻止儿童看到）？有些域名可以自我组织，而另一些则对谁可以得到一个该域名空间中的名字有限制，正如图 7-2 中所指出的那样。但是，什么样的限制才是适当的呢？我们用 pro 域名作为例子说明。这是一个分配给合格的专业人员所用的名字。但问题是谁才是专业的呢？显然医生和律师是专业人士，但自由摄影师、钢琴教师、魔术师、管道工、理发师、灭虫者、纹身艺术家和雇佣军等算专业人士吗？这些职业有资格当选吗？按照谁的标准呢？

域	预定使用	启用时间	限制否
com	商业	1985	不限
edu	教育科研	1985	限
gov	政府	1985	限
int	国际组织	1988	限
mil	军事	1985	限
net	网络提供商	1985	不限
org	非营利组织	1985	不限
aero	航空运输	2001	限
biz	公司	2001	不限
coop	合作	2001	限
info	信息	2002	不限
museum	博物馆	2002	限
name	人	2002	不限
pro	专业	2002	限
cat	加泰罗尼亚	2005	限
jobs	就业	2005	限
mobi	移动设备	2005	限
tel	联络资料	2005	限
travel	旅游业	2005	限
×××	色情业	2010	不限

图 7-2 通用的顶级域名

域名还与金钱挂钩。图瓦卢（国）将它的域名 tv 出售，获得租赁权 50 万美元，就是因为这个国家的域名非常适合广告电视网站。实际上，com 域名下几乎采用了每一个普通词（英文），而且这些词有最常见的拼写错误。家居用品、动物、植物以及身体各部位等名字都被人尝试注册了域名。注册所有人一转身就以高得多的价格把域名出售给感兴趣的人，而所有的一切都只是名字而已。这就是所谓的域名抢注（cybersquatting）。当 Internet 时代来临时，许多公司起跑得比较缓慢，以至于突然发现显而易见应该是自己的域名已经被他人注册了，于是它们试图收购这样的名字。在一般情况下，只要商标没有受到侵犯，而且没有涉及欺诈，那么域名的授予是先到先得的。然而，用以解决域名纠纷的政策仍在不断细化之中。

每个域的名字是由它向上到（未命名的）根节点的路径来命名的，路径上的各个部分用句点（读作“dot”）分开。因此，Cisco 公司的工程部门可能是 eng.cisco.com.，而不是像

UNIX 风格的名字/com/cisco/eng。请注意，这种层次的命名机制意味着 eng.cisco.com. 中的 eng 不会与 eng.washington.edu. 中的 eng 发生冲突，华盛顿大学的英语系可能会取它作为自己的域名。

域名可以是绝对的，也可以是相对的。绝对域名总是以句点作为结束（例如 eng.cisco.com.），而相对域名则不然。相对域名必须在一定的上下文环境中被解释才有的真正含义。无论是绝对域名还是相对域名，一个域名对应于域名树中一个特定的结点，以及它下面的所有结点。

域名不区分大小写，因此，edu、Edu 和 EDU 的含义都一样。各组成部分的名字最多可以有 63 个字符，整个路径的名字不得超过 255 个字符。

原则上，域名可以被插入到域名树中的通用域名空间或者国家域名空间。例如，cs.washington.edu 完全可以被列在国家域名 us 的下面，从而变成 cs.washington.wa.us。然而，实际上美国的大多数组织都位于某一个通用域名下面，而美国之外的大多数组织则位于其国家域名的下面。没有规则不允许一个组织在多个顶级域下注册域名。大型公司通常就这么做（例如 sony.com 和 sony.nl）。

每个域自己控制如何分配它下面的子域。例如，日本的 ac.jp 和 co.jp 域分别对应于 edu 和 com；但荷兰不做这样的区分，它把所有的组织直接放在 nl 下。因此，以下 3 个域名都表示大学的计算机科学系：

- (1) cs.washington.edu（美国华盛顿大学）。
- (2) cs.vu.nl（荷兰阿姆斯特丹自由大学）。
- (3) cs.keio.ac.jp（日本庆应义塾大学）。

为了创建一个新域，创建者必须得到包含该新域的上级域的许可。例如，如果华盛顿大学建立了一个 VLSI（超大规模集成电路）组，并且希望将它命名为 vlsi.cs.washington.edu，那么这个组必须获得管理 cs.washington.edu 域名的管理员许可。类似地，如果创立了一所新的大学，比如说 University of Northern South Dakota，那么它必须请求负责 edu 域名的管理员将 unsd.edu 分配给它。按照这种方式分配域名就可以避免名字冲突，并且每个域都可以跟踪它所有的子域。一旦创建并注册了一个新域，则该新域就可以创建属于自己的子域，比如 cs.unsd.edu，而无须得到域名树中任何上层域的许可。

命名机制遵循的是以组织为边界，而不是以物理网络为边界。例如，即使计算机科学系和电子工程系在同一幢楼里，并使用同一个 LAN，但它们仍然可以属于完全不同的域。同样地，即使计算机科学系分散在 Babbage Hall 和 Turing Hall 这两幢楼里，但这两幢楼里的主机通常仍属于同一个域。

7.1.2 域名资源记录

无论是只有一台主机的域还是顶级域，每个域都有一组与它相关联的资源记录（resource record）。这些记录组成了 DNS 数据库。对于一台主机来说，最常见的资源记录就是它的 IP 地址，但除此以外还存在着许多其他种类的资源记录。当解析器把一个域名传递给 DNS 时，它能获得的 DNS 返回结果就是与该域名相关联的资源记录。因此，DNS 的基本功能是将域名映射到资源记录。

一条资源记录是一个五元组。尽管出于效率考虑资源记录被编码成了二进制的形式，但是大多数说明性资料还是用 ASCII 文本来表示资源记录，每一条记录占一行。在接下来的介绍中，我们将使用如下的格式：

```
Domain_name Time_to_live Class Type Value
```

Domain_name（域名）指出了这条记录适用于哪个域。通常每个域有许多条记录，并且数据库的每份副本保存了多个域的信息。因此 **Domain_name** 是匹配查询条件的主要搜索关键字。数据库中资源记录的顺序则无关紧要。

Time_to_live（生存期）指明了该条记录的稳定程度。极为稳定的信息会被分配一个很大的值，比如 86 400（1 天时间里的秒数）；而非常不稳定的信息则会被分配一个较小的值，比如 60（1 分钟的秒数）。稍后当我们讨论缓存机制时将再回到这个议题。

每条资源记录的第三个字段是 **Class**（类别）。对于 Internet 信息，它总是 IN。对于非 Internet 信息，则可以使用其他的代码，但实际上很少见。

Type（类型）字段指出了这是什么类型的记录。DNS 记录有许多类型，图 7-3 列出了最重要的一些类型。

类型	含义	值
SOA	授权开始	本区域的参数
A	主机的IPv4地址	32位整数
AAAA	主机的IPv6地址	128位整数
MX	邮件交换	优先级，愿意接受邮件的域
NS	域名服务器	本域的服务器名字
CNAME	规范名	域名
PTR	指针	IP地址的别名
SPF	发送者的政策框架	邮件发送政策的文本编码
SRV	服务	提供服务的主机
TXT	文本	说明的ASCII文本

图 7-3 主要的 DNS 资源记录类型

SOA 记录给出了有关该名字服务器区域（后面将会介绍）的主要信息源名称、名字服务器管理员的电子邮件地址、一个唯一的序号以及各种标志位和超时的值。

最重要的记录类型是 A（地址）记录，它包含了某台主机一个网络接口的 32 位 IP 地址。对应的 AAAA，或“quad A”记录包含了一个 128 位的 IPv6 地址。每台 Internet 主机必须至少有一个 IP 地址，以便其他机器能与它进行通信。某些主机有两个或多个网络接口，在这种情况下，它们就有两个或多个 A 或 AAAA 资源记录。因此，对于单个域名的查询可能获得多个地址。

最常用的记录类型是 MX 记录，它指定了一台准备接受该特定域名电子邮件的主机的名字。因为并非每台机器都做好了接收电子邮件的准备，因此必须用一个记录作特别说明。例如，如果有人打算发送电子邮件给某个人，比如 bill@microsoft.com，则发送主机必须找到在 microsoft.com 中愿意接收电子邮件的邮件服务器。MX 记录就是用来提供这样的信息。

另一个重要记录类型是 NS 记录。它指明了一台用于所在域和子域的名字服务器。这是一台拥有一份某域数据库副本的主机。这条记录被用于域名查询处理。稍后我们将对此

做简单的讨论。

CNAME 记录允许创建别名。例如，如果一个人很熟悉 Internet 的常规命名规则，他打算给 MIT 计算机科学系名叫 paul 的人发送邮件，他就猜测此人的邮箱名是 paul@cs.mit.edu。实际上，这个地址不正确，因为 MIT 计算机科学系的域名是 csail.mit.edu。但是，MIT 可以创建一条 CNAME 记录指向人和程序的正确方向，为那些不知情的人提供一项服务。类似下面这样的一条记录就可以完成此任务：

```
cs.mit.edu 86400 IN CNAME csail.mit.edu
```

如同 CNAME 一样，PTR 指向另一个名字。但是 CNAME 只是一个宏定义（即可以用另一个串来替代一个串的机制），而 PTR 与 CNAME 不同，它是一种正规的 DNS 数据类型，它的确切含义取决于所在的上下文。实际上，PTR 几乎总是被用来将一个名字与一个 IP 地址关联起来，以便能够通过查询 IP 地址来获得对应机器的名字，这种功能称为逆向查询（reverse lookups）。

SRV 是一个新的记录类型，它把主机标识为域内的一种给定服务。例如，cs.washington.edu 的 Web 服务器可以标识成 cockatoo.cs.washington.edu。这个记录能产生 MX 记录，并执行相同的任务，但只适用于邮件服务器。

SPF 也是一个新的记录类型。它可以让一个域把邮件服务信息进行编码，即域内哪台机器负责把邮件发送到 Internet 其余地方。这条记录有助于接收机器检查邮件是否有效。如果接收到的邮件来自一台通话本身躲躲闪闪的机器，但域名记录说明邮件只能来自域外一台称为 smtp 的机器，那么该邮件就是伪造的垃圾邮件。

列表的最后一行给出了 TXT 记录，这是最初为了允许域以任意方式标识自己而提供的一种途径。如今，它们通常被编码成机器可读信息，一般是 SPF 信息。

最后，资源记录还包括 Value 字段，该字段的值可以是一个数字、一个域名或者一个 ASCII 字符串，其语义取决于记录的类型。图 7-3 中给出了每种主要记录类型的 Value 字段的简短描述。

图 7-4 显示的例子给出了在一个域的 DNS 数据库中可能找到的信息种类。它描述了图 7-1 中 cs.vu.nl 域（假设的）数据库的一部分。该数据库包含 7 种资源记录。

图 7-4 中第一个非注释行给出了该域的一些基本信息，我们以后将不再关心这些信息。接下来的两个表项给出了第一个和第二个投递电子邮件给 person@cs.vu.nl 的地点。首先尝试的是 zephyr（一台特定的机器），如果给它发送失败了，则下一个应该尝试给 top 发送。下一行标识了该域的名字服务器为 star。

接下来是一个空白行，加入空白行的目的是为了增加可读性。再下面的几行给出了 star、zephyr 和 top 的 IP 地址。紧跟这些行后面的是别名 www.cs.vu.nl，因此可以使用这个地址而无须指派一台特殊的机器。创建这个别名的好处是允许 cs.vu.nl 改变它的 WWW 服务器，无须使得人们原来所使用的地址失效。类似地，ftp.cs.vu.nl 也是一个别名。

有关机器 flits 的信息部分列出了两个 IP 地址和处理发送至 flits.cs.vu.nl 电子邮件的三种选择。首先选择的是 flits 本身，但是如果它失效，zephyr 和 top 是第二个和第三个选择。

; cs.vu.nl的授权数据					
cs.vu.nl	86 400	IN	SOA	star boss (9527, 7200, 7200, 241 920, 86 400)	
cs.vu.nl.	86 400	IN	MX	1 zephyr	
cs.vu.nl.	86 400	IN	MX	2 top	
cs.vu.nl.	86 400	IN	NS	star	
star	86 400	IN	A	130.27.56.205	
zephyr	86 400	IN	A	130.37.20.10	
top	86 400	IN	A	130.37.20.11	
www	86 400	IN	CNAME	star.cs.vu.nl	
ftp	86 400	IN	CNAME	zephyr.cs.vu.nl	
flits	86 400	IN	A	130.37.16.112	
flits	86 400	IN	A	192.31.231.165	
flits	86 400	IN	MX	1 tlits	
flits	86 400	IN	MX	2 zepnyr	
flits	86 400	IN	MX	3 top	
rowboat		IN	A	130.37.56.201	
		IN	MX	1 rowboat	
		IN	MX	2 zephyr	
little-sister		IN	A	130.37.62.23	
laserjet		IN	A	192.31.231.216	

图 7-4 针对 cs.vu.nl 域名可能的 DNS 数据库一部分

接下来的 3 行包含了一台计算机的典型表项，在这个例子中是 rowboat.cs.vu.nl。它提供的信息包括 IP 地址、主要邮件存放处和次要邮件存放处。然后是一个针对一台计算机的表项，这台计算机自己不能接收邮件；紧随其后的一个表项可能是指一台连接到 Internet 的打印机。

7.1.3 域名服务器

至少在理论上，一台域名服务器就可以包含整个 DNS 数据库，并响应所有对该数据库的查询。但实际上，这台服务器会因负载过重而变得毫无用处。而且，一旦它停机，则整个 Internet 将会瘫痪。

为了避免由于单个信息源带来的各种问题，DNS 名字空间被划分为一些不重叠的区域 (zones)。针对图 7-1 的名字空间，一种可能的划分方法如图 7-5 所示。每个圈起来的区域包含域名树的一部分。

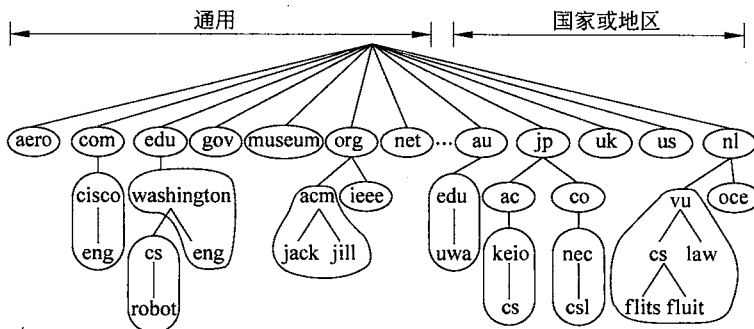


图 7-5 按区域划分（圈起来的）的部分 DNS 名字空间

区域边界应该放置在区域中的什么位置由该区域的管理员来决定。这个决定在很大程度上取决于需要在哪里使用多少个名字服务器。例如，在图 7-4 中，华盛顿大学有一个区

域 `washington.edu`，它要处理 `eng.washington.edu` 但不能处理 `cs.washington.edu`，这是另一个区域，有它自己的域名服务器。当有些系（比如英语系）不希望运行自己的域名服务器，而另外一些系（比如计算机科学系）却希望运行自己的域名服务器时，就有可能作出这样的决定。

每个区域都与一个或多个域名服务器关联。这些服务器是持有该区域数据库的主机。通常情况下，一个区域有一个主域名服务器和一个或多个辅域名服务器。主服务器从自己磁盘的一个文件读入有关域名信息，辅域名服务器从主域名服务器获取域名信息。为了提高可靠性，一些域名服务器可以设置在区域外面。

查询一个名字和找出其对应地址的过程称为域名解析（name resolution）。当解析器需要查询一个域名，它就把该查询传递给一个本地域名服务器。如果需要寻找的域恰好落在该域名服务器管辖下，比如 `top.cs.vu.nl` 在 `cs.vu.nl` 的管辖下，则该域名服务器就返回权威资源记录。一个权威记录（authoritative record）由管理该记录的权威部门提供，因此总是正确的。权威记录的权威性是相对缓存记录（cached record）而言的，缓存的记录有可能过时了。

如果被查询域在远端，比如 `flits.cs.vu.nl` 要找到华盛顿大学（UW）`robot.cs.washington.edu` 的 IP 地址，会发生什么？在这种情况下，如果本地没有关于相关域的缓存信息，那么域名服务器启动一次远程查询。该查询遵循如图 7-6 所示的过程。第 1 步，显示查询报文被发送到本地域名服务器。查询中包含了被查询的域名、类型（A）和类（IN）。

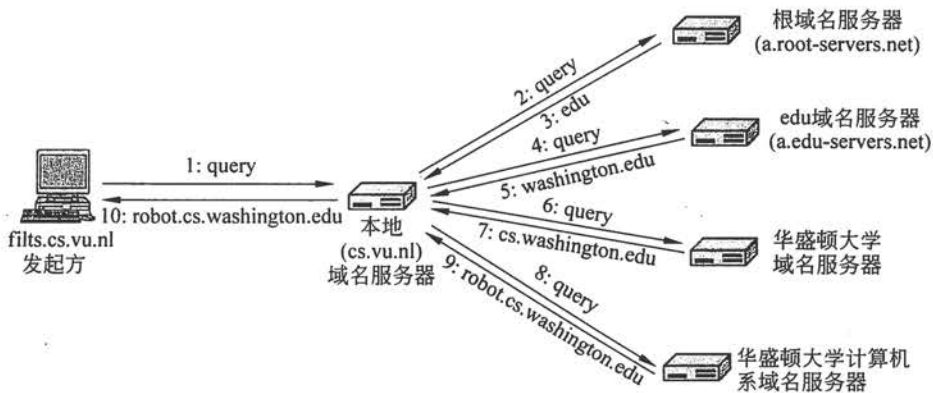


图 7-6 解析器查询一个远程名字的 10 个步骤

下一步是通过请求其中之一的根域名服务器来启动域名层次结构顶部的查询。这些域名服务器包含每个顶级域名的有关信息。这显示在图 7-6 中的第 2 步。为了与一个根服务器取得联系，每个域名服务器必须有一个或多个根域名服务器的信息。这个信息通常放在一个系统配置文件中，在 DNS 服务器启动时把该文件加载到 DNS 缓存。这个文件很简单，只是列出了关于根服务器的 NS 记录和相应的 A 记录。

总共有 13 个根 DNS 服务器，毫无创意地被命名为从 `a.root-servers.net` 到 `m.root-servers.net`。每个根服务器可以是逻辑上的一台计算机。然而，由于整个 Internet 依赖于根服务器，因此它们必须是能力超强的，并且被大量复制的计算机。大多数服务器被放置在多个地理位置，查询报文通过选播路由到达其中一台服务器。选播是一种特殊的路由，它能将数据包路由到最近的一个目标地址实例。我们在第 5 章中描述了选播路由。复制技术

能提高域名系统的可靠性和性能。

根域名服务器不可能知道在 UW 的一台机器的地址,可能还不知道 UW 的域名服务器。但是,它必须知道 edu 域的域名服务器,因为 cs.washington.edu 属于 edu 域管辖下。在第 3 步,它返回查询的答案,其中包括了名字和 IP 地址。

然后本地域名服务器继续尽职地执行任务。它将整个查询发给 edu 域名服务器(a.edu-servers.net)。该域名服务器返回 UW 的域名服务器。上述过程显示在图中的第 4 步和第 5 步。现在,快接近目标了,本地域名服务器把查询发送给 UW 的域名服务器(第 6 步)。如果被查询的域名是英语系,则马上能找到答案,因为 UW 域包括了英语系。但是,计算机科学系选择运行自己的域名服务器,因此该查询返回的是 UW 计算机科学系的域名服务器和 IP 地址(第 7 步)。

最后,本地域名服务器查询 UW 计算机科学系的域名服务器(第 8 步)。这台服务器是 cs.washington.edu 的权威机构,所以它必定有答案。它返回最终的答案(第 9 步),即作为响应 flits.cs.vu.nl(第 10 步)转发的本地域名服务器。至此,查询的域名得到了解析。

你可以用标准的工具来探寻整个查询过程,比如安装在大多数 UNIX 系统中的 dig 程序。例如,键入:

```
dig@a.edu-servers.net robot.cs.washington.edu
```

向 a.edu-servers.net 域名服务器发出一个针对 robot.cs.washington.edu 的查询,并打印出查询结果。这个结果显示了上面例子中第 4 步获得的信息,你将了解到 UW 域名服务器的名字和 IP 地址。

有关这种长距离远程查询的场景,有 3 个技术要点值得讨论。首先,在图 7-6 中两个不同的查询机制在工作。当主机 flits.cs.vu.nl 将查询发送给本地域名服务器后,该域名服务器就代替该主机处理域名解析工作,直到它返回所需的答案。这里的答案必须是完整的,它不能返回部分答案。虽然部分答案可能也会有所帮助,但不是查询应该得到的结果。这个机制称为递归查询(recursive query)。

另一方面,根域名服务器(和每个后续的域名服务器)并不是递归继续查询本地域名服务器。它只是返回一个部分答案,并移动到下一个查询操作。本地域名服务器负责继续解析,具体做法是发出进一步的查询报文。这个机制称为迭代查询(iterative query)。

一次域名解析可以涉及这两种机制,如同该例子所表明的那样。递归查询似乎总是最可取的,但许多域名服务器(尤其是根服务器)并没有采用这种处理方式,它们太忙了。迭代查询则将重负压在了查询发起方。本地域名服务器支持递归查询的理由是它负责为其域内的主机提供服务。这些主机不必配置成运行一个完整的域名服务器,它们只要刚好能到达本地域名服务器即可。

值得讨论的第二点是缓存。所有的查询答案,包括所有的部分答案都会被缓存。这样,如果另一台 cs.vu.nl 主机需要查询 robot.cs.washington.edu,那么答案早就有了。更妙的是,如果在同一个域中的一台不同主机需要查询另一台主机,比如 galah.cs.washington.edu,那么此次查询可直接发送到权威的域名服务器。类似地,针对 washington.edu 其他域的查询也可以从 washington.edu 域名服务器直接开始。这种使用缓存答案的做法可大大降低一次查询的步骤,并能提高查询性能。事实上,我们在例子中勾勒出的原始场景是最坏的情况,

通常发生在没有任何缓存有用信息时。

然而，高速缓存的答案不具权威性，因为在 `cs.washington.edu` 所做的信息变化将不会传播到世界上所有可能知道它的缓存。出于这个原因，缓存的表项不应该生存得太长。这就是为什么在每条资源记录中要包含 `Time_to_live` 字段的原因。它告诉远程域名服务器可缓存本记录多长时间。如果某台机器已经多年使用相同的 IP 地址，将它的信息缓存 1 天可能是安全的。而对于波动比较大的信息，几秒钟或一分钟后清除掉记录的做法或许更安全。

第三个讨论的问题关于查询和响应使用的传输层协议。域名系统采用的是 UDP。DNS 消息通过 UDP 数据包发送，格式非常简单，只有查询和响应；域名服务器可用此数据包继续进行解析操作。我们不讨论这种报文格式的细节。如果在很短的时间内没有响应返回，DNS 客户端必须重复查询请求；如果重复一定次数后仍然失败，则尝试域内另一台域名服务器。查询过程这样设计的主要目的是为了应付出现服务器关闭以及查询或响应包丢失的情况。每个查询报文都包含 16 位的标识符，这个标识符将被复制到响应包中，以便域名服务器将接收到的答案与相应的查询匹配，即使它同时发出了多个查询也不会弄混查询结果。

即使 DNS 的目的很简单，人们应该明确这是一个庞大而复杂的分布式系统，它由数百万计的域名服务器组成，这些服务器要一起协同工作。DNS 在人类可读域名和机器 IP 地址之间形成了一个关键的衔接。为了提高性能和可靠性，它还利用了复制和缓存机制，同时设计得具有很强的鲁棒性。

我们没有涉及任何安全，但正如你可能想象的那样，如果心怀恶意，那么拥有改变域名到地址映射的能力就可能造成灾难性的后果。出于这个原因，研究者们已经开发了专用于 DNS 的安全扩展，这个安全机制称为 DNSSEC。我们将在第 8 章介绍这些内容。

应用程序对域名的使用方式希望更加灵活化，这样的应用需求逐渐显露。例如，首先对内容进行命名，然后解析出拥有该内容的附近一个主机的 IP 地址。这种映射模式特别符合搜索一个电影并下载它的应用模式。这时，人们关注的是电影本身，而不是某个拥有电影拷贝的计算机，因此解析所要的全部结果只是附近具有该影片拷贝的任何一台计算机 IP 地址。内容分发网络就是完成这个映射的一种实现方式。我们将在 7.5 节介绍如何利用本章后面的 DNS 来构建这样的网络。

7.2 电子邮件

电子邮件或者更常用的 E-mail，已经存在 30 多年了。由于比纸质信件更快更便宜，电子邮件成为自早期 Internet 出现以来最广泛的应用。在 1990 年以前，它主要被用于学术界。在整个 20 世纪 90 年代，它变得普及起来并呈指数形式增长，以至于现在每天发送的电子邮件数量远远超过了传统的纸质邮件（snail mail）数量。其他形式的网络通信，比如即时消息和 IP 语音在近 10 年也有了极大的发展，但是电子邮件仍然是 Internet 通信的主要负载。电子邮件广泛地用于业界公司内部通信，例如，分散在世界各地的员工们就一个复杂项目进行协同。不幸的是，像纸质信件一样，大多数电子邮件都是邮寄宣传品或者垃圾邮件（spam），某些甚至 10 封邮件里面高达 9 封是垃圾邮件（McAfee, 2010）。

与大多数其他通信方式一样，电子邮件有它自己的约定和风格。特别是它非常不拘小

节，使用户门槛很低。那些从来没有想过给某个大人物打一个电话或者写一封信的人，可以毫不犹豫地给他发一封随意书写的电子邮件。由于消除了与社会地位、年龄和性别有关的大多数暗示，通过电子邮件展开辩论通常关注于内容本身而不是状态。有了电子邮件，某个暑期学生的绝妙想法比一个执行副总裁的愚钝想法影响力更大。

电子邮件中充满了诸如 BTW (By The Way, 顺便)、ROTFL (Rolling On The Floor Laughing, 笑得满地打滚) 和 IMHO (In My Humble Opinion, 恕我直言) 等行话。很多人还在他们的电子邮件中使用一些称为**微笑图标** (smiley) 的 ASCII 符号，这种符号始于普适的 “:-)”。如果你不熟悉这个符号，那么把本书旋转 90° 就能明白。这种符号和其他情绪图标 (emoticon) 有助于传递邮件语气，现在它们已经被扩散到其他形式的通信中，比如即时消息。

电子邮件协议在其使用期间经历了很大的演变。第一个电子邮件系统简单地由文件传输协议和约定组成，约定规定每个邮件的第一行 (即文件) 必须给出收件人地址。随着时间的推移，文件传输和电子邮件分歧越来越大，最终电子邮件从文件传输中分离出来并增加了许多功能，例如发送一个邮件给一组收件人的能力。在 20 世纪 90 年代，多媒体功能变得非常重要，邮件可以包括图像和其他非文字材料。相应地，阅读电子邮件的程序也变得更为复杂，从单纯的基于文本阅读转变成图形用户界面，并且为用户增加了在任何地方通过笔记本电脑访问邮件的能力。最后，随着垃圾广告邮件的盛行，邮件阅读器和邮件传输协议现在必须具备发现这些不想要的邮件并删除它们的能力。

在我们的电子邮件描述中，我们将精力集中在用户之间如何移动邮件，而没有考查和体验邮件阅读程序。然而，在描述了整体架构后，我们将开始介绍电子邮件系统中面向用户的那一部分，因为它是为大多数读者所熟悉。

7.2.1 体系结构和服务

在本节，我们将提供一个概述，说明电子邮件系统如何组织以及它们可以做什么。电子邮件系统的体系结构如图 7-7 所示。它包括两类子系统：**用户代理** (user agent) 和**邮件传输代理** (message transfer agent)。人们通过用户代理阅读和发送电子邮件；邮件传输代理负责将用户邮件从源端移动到目的地。我们把邮件传输代理非正式地称为**邮件服务器**。

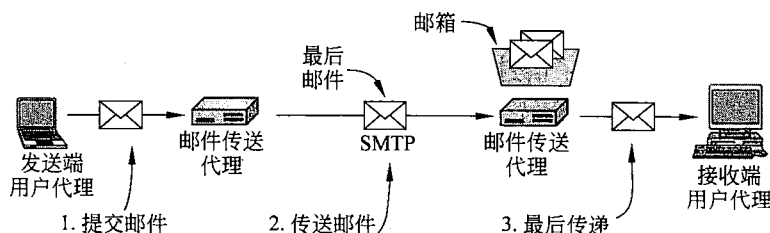


图 7-7 电子邮件系统的体系结构

用户代理是一个程序，用户通过它与电子邮件系统交互。用户代理提供了一个图形界面，有时是一个基于文本和基于命令的接口。它包括了撰写邮件、回复邮件、显示入境邮件信息的手段，同时还提供了如何过滤、搜索和删除邮件的组织方式。把新邮件发送给邮件系统，并通过它传递的行为称为**邮件提交** (mail submission)。

有些用户代理可能会自动完成对邮件的处理，预测用户想要什么。例如，为了提取出或者降低可能是垃圾邮件的优先级，入境邮件可能会先被过滤。某些用户代理还包括一些先进功能，比如安排电子邮件的自动回复（“现在我正在度一个美妙假期，一旦我回去将立即给你回复”）。用户代理运行在用户阅读邮件的同一台计算机上。这只是另一个程序，而且或许只运行一段时间。

邮件传输代理通常是系统进程。它们运行在邮件服务器机器的后台，并始终保持运行状态。它们的工作是通过系统自动将电子邮件从发送端移动到收件人，采用的协议是简单邮件传输协议（SMTP，Simple Mail Transfer Protocol）。这是邮件传输的必经之路。

SMTP 最早由 RFC 821 说明，之后被修订成为当前的 RFC 5321。它通过一个连接发送邮件、返回传递状态和任何错误的报告。对许多应用来说，确认交付非常重要，甚至可能具有法律上的意义（“哦，先生，我的电子邮件系统没那么可靠，所以我猜测电子传票可能遗失在某个地方了”）。

邮件传输代理还实现了邮件列表（mailing list）功能，一个邮件的完全相同副本被传递到电子邮件地址列表中的每个人。其他先进的功能包括抄送、秘密抄送、高优先级电子邮件、秘密（即加密）电子邮件；如果主要收件人当前不方便接收邮件，那么可指定另一个接收者，以及阅读老板邮件并代替回复邮件的辅助能力。

将用户代理和邮件传输代理衔接起来的是邮箱这个概念，以及电子邮件的标准格式。邮箱（mailbox）存储用户收到的电子邮件。邮箱由邮件服务器负责维护，用户代理只需向用户展示邮箱中的内容即可。要做到这一点，用户代理向邮件服务器发送操纵邮箱的命令，包括检查邮箱内容、删除邮件等。邮件检索在图 7-7 中是最后交付（第 3 步）。在这样的体系结构下，一个用户可以在多台计算机上使用不同的用户代理来访问同一个邮箱。

在两个邮件传输代理之间发送的邮件具有标准的格式。原来由 RFC 822 规定的格式已被修订为当前的 RFC 5322，并且扩展成支持多媒体内容和国际文本。这个方案称为 MIME，稍后将对此进行讨论。虽然，人们仍然将 Internet 电子邮件看作 RFC 822。

电子邮件系统的一个关键思想是将信封（envelope）与邮件内容区分开来。信封将消息封装成邮件，它包含了传输消息所需要的所有信息，例如目标地址、优先级和安全等级，所有这些都有别于消息本身。消息传输代理根据信封来进行路由，就好像邮局的做法一样。

信封内的消息由两部分组成：邮件头（header）和邮件体（body）。邮件头包含用户代理所需的控制信息。邮件体则完全提供给收件人，代理和邮件传输代理都不在意邮件体包含了什么信息。图 7-8 中显示了信封和邮件。

下面，我们按照一个用户给另一个用户发送电子邮件涉及的每个步骤，来详细考查这种体系结构下的每个组成部分。这个考查之旅就从用户代理开始。

7.2.2 用户代理

用户代理是一个程序（有时也称为电子邮件阅读器），它接收各种各样命令，从接收和回复邮件到操纵邮箱的命令。目前流行的用户代理有许多，其中包括谷歌 Gmail、微软的 Outlook、Mozilla 的 Thunderbird 和苹果的 Apple Mail。这些用户代理在外形上相差很大。大多数用户代理有一个菜单或图标驱动的图形界面，需要使用鼠标进行操作；在小型移动

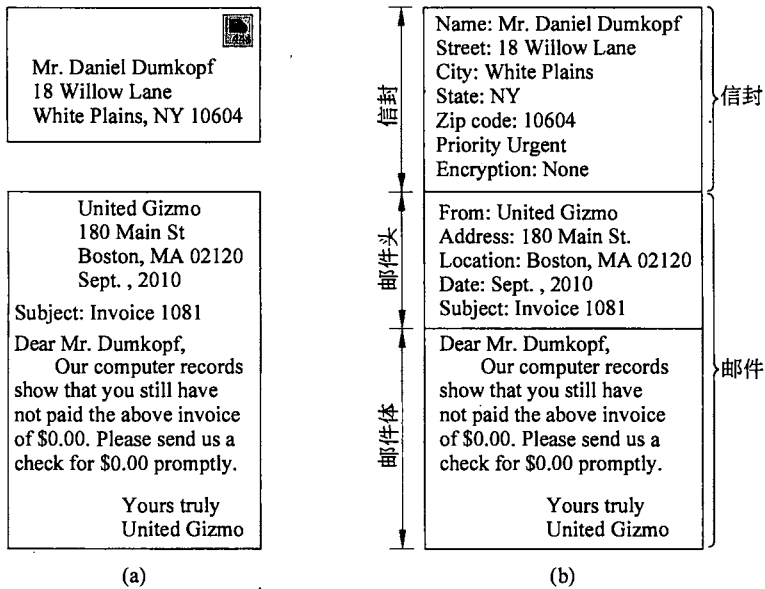


图 7-8 信封和邮件
(a) 纸质信件; (b) 电子邮件

设备上的界面通常是触摸界面。老的用户代理，比如 Elm、mh 和 Pine 提供的是基于文本的界面，期望用户从键盘输入一个字符命令。从功能上看，这些界面都相同，至少对文本消息是一样的。

用户代理界面的典型元素如图 7-9 所示。你的邮件阅读器可能比这个要华丽得多，但或许具有同等的功能。当用户代理被启动时，它一般会给出用户邮箱内邮件的摘要信息。通常情况下，摘要信息是每个邮件占一行，并且按照某种排序排列。摘要突出的是从邮件信封或邮件头中提取出来的一些关键字段。

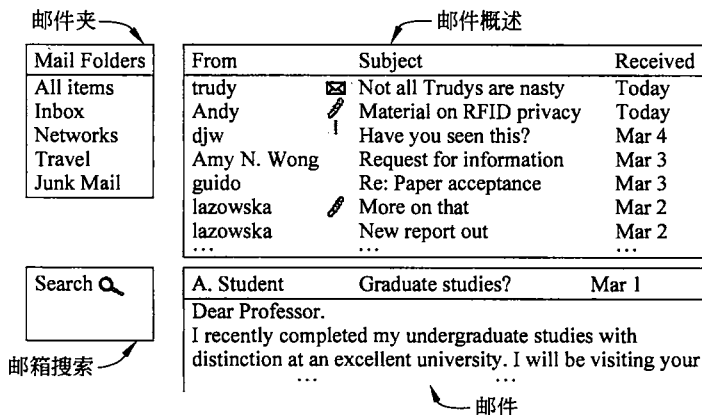


图 7-9 用户代理接口的典型元素

图 7-9 显示了的例子中的摘要信息有 7 行。每一行使用了发件人、标题和何时接收三个字段，显示的顺序可以按照邮件的发送者、邮件的标题以及邮件的接收时间来排列。所有信息的格式都以用户友好的方式呈现，而不是显示消息字段的文字内容，但无论采用什么方式都必须基于邮件字段。因此，如果人们在发送邮件时不包括标题字段，那么他们常

常会发现针对该电子邮件的回复通常得不到最高优先级。

摘要中也可能包括许多其他字段或指示信息。例如，图 7-9 中邮件标题旁边的图标可能表明未读邮件（信封）、有附件（回形针）和重要邮件，至少发件人判定为重要（感叹号）。

还可以按照其他规则进行邮件的排序。最常见的是根据接收到邮件的时间，将最近收到的邮件优先显示，同时还使用一些指示消息是新的还是已被用户读取过的图标。在邮件摘要显示的字段和排序方式用户可以自己的喜好定制。

需要时用户代理必须能够显示入境邮件信息，方便人们阅读他们的电子邮件。通常系统会提供一个短消息预览，如图 7-9 所示，帮助用户决定何时进一步阅读该邮件。预览可以使用小图标或图像来描述邮件的内容。用户代理还提供其他表示的处理方式，包括重新格式化邮件来适应当前的显示，或者翻译或转换邮件内容成更便利的显示格式（例如，可识别文本的数字化语音）。

邮件被阅读过后，用户可以决定用它来做什么。这就是所谓的邮件处置（message disposition）。邮件的进一步处理选项包括删除邮件、回复邮件、把邮件转发给另一个用户，以及保存邮件供日后参考。大多数用户代理可以用多个文件夹保存一个邮箱的入境邮件。文件夹允许用户根据发件人、标题或一些其他分类方法来保存消息。

在用户阅读邮件之前，用户代理可以自动完成邮件的归档。一个常见的例子，系统检查邮件的字段和内容，根据有关以前邮件的用户反馈，使用邮件字段和内容来确定是否可能是垃圾邮件。许多 ISP 和企业都运行这样的软件，给邮件贴上重要或者垃圾邮件的标签，这样用户代理可以将这些邮件归档到相应的邮箱中。对于许多用户来说，ISP 和公司具有预先看到邮件的优势，并且它们可能拥有已知垃圾邮件发送者的名单。如果几百个用户恰好刚刚收到类似的消息，那么这个邮件很有可能是垃圾邮件。通过预先将入境邮件分类到“可能合法”和“可能垃圾邮件”的做法，用户代理可以为用户节省从垃圾邮件中分离出有用信息的大量工作。

那么，什么是最流行的垃圾邮件？它是由一组被感染的计算机产生的邮件，具体内容取决于你居住的地方。这种被感染的计算机集合就称为僵尸网络（botnet）。在亚洲，典型的垃圾邮件是假文凭的制作；在美国，廉价药品和其他可疑产品的供给是典型的垃圾邮件；而有关无人认领的尼日利亚银行账户比比皆是；增强身体各部位的假药则无处不在。

用户还可以构造其他归档规则。每个规则指定了一个条件和相应的一个动作。例如，规则可以将来自老板的邮件都放到一个要立即阅读的文件夹中，而指定从某个特定邮件列表发来的任何消息存到另一个文件夹供稍后阅读。图 7-9 显示了几个文件夹。其中最重要的文件夹是 Inbox（收件箱），用来存放没有预先规定存放的邮件，即没有其他地方可去的入境邮件；还有一个文件夹是 Junk Mail（垃圾邮件），被认为是垃圾邮件的邮件都归类到该文件夹。

除了显式构造自己喜欢的各种文件夹外，用户代理还为用户提供了丰富的搜索邮箱功能。此功能如图 7-9 所示。搜索能力为用户提供了快速查找邮件的手段，比如上个月有人发送的有关“在哪里买 Vegemite”的邮件。

电子邮件自从刚出现时作为一种文件传输以来，已经走过了很长的路。现在服务提供商经常能为用户支持高达 1 GB 的邮箱用来存储邮件，而且这些邮件是用户在很长一段时间内邮件交互的细节。用户代理先进的邮件处理的能力，包括搜索和自动分类处理，使得

它可以管理这些大量的电子邮件。对于一年要发送和收到数千封电子邮件的用户来说，这些工具非常宝贵。

另一个有用的功能是用代理能够以某种方式自动响应邮件。一种响应是将入境的电子邮件转发到一个不同的地址，例如，由一个商业传呼服务所操纵的计算机，通过无线电或卫星寻呼用户，并在他的寻呼机上显示“主题：”行。这些自动响应功能必须运行在邮件服务器上，因为用户代理可能无法在所有的时间内都保持运行状态，而且很有可能只是偶尔检索电子邮件。基于这些因素的考虑，用户代理不能提供一个真正的自动响应服务。然而，自动响应的接口通常由用户代理表示。

另一个不同的自动响应例子是**休假代理 (vacation agent)**。这是一个程序，它首先检查每个入境消息，然后给发件人回送一个平淡的答复，比如：“嗨。我在度假。我将于 8 月 24 日回来后立即联系你。”这样的答复也可以指定如何处理在此期间发生的紧急事项，或者针对某人就某个特殊问题的邮件给予回复等。大多数度假代理跟踪它们已经给哪些用户发过这种“罐装回复”(canned reply)，以免给同一人重复发送相同的回复。然而，这些代理依然有缺陷。例如，它给来自一个大邮件列表的群发邮件回送一个罐装答复显然是不可取的。

让我们现在把注意力转到一个用户将消息发送给另一个用户的场景。用户代理支持的基本功能之一是邮件的组成，这点我们还没有讨论过。邮件的组成涉及创建邮件、回复邮件，以及为了将邮件交付给收件人而将这些邮件发送到邮件系统的其余部分。虽然在创建邮件时，可以使用任何文本编辑器来撰写邮件正文，但邮件正文编辑器通常集成在用户代理中，以便它可以为每个邮件提供类似寻址和多个头字段的辅助功能。例如，在回复邮件时，电子邮件系统可以从收到的邮件中提取发件人的地址，并自动插入到回复邮件的适当的地方。其他一些共同功能包括在邮件底部追加一个**签名块 (signature block)**、更正拼写错误和计算表明该消息是否有效的数字签名。

发往邮件系统的邮件遵循一个标准格式，这个格式的创建基于提供给用户代理的信息。传输消息的最重要部分是信封，信封中最重要的是目的地址。邮件目的地址必须是邮件传输代理可以处理的格式。

期望的地址形式是用户@DNS 地址 (user@dns-address)。由于我们已经在本章前面研究过 DNS，我们在这里就不再赘述。然而，值得注意的是还存在其他形式的地址。特别是，X.400 地址看上去和 DNS 地址完全不同。

X.400 是邮件处理系统的 ISO 标准，它曾经是 SMTP 的竞争对手。SMTP 轻而易举地取得了胜利，虽然仍有人在使用 X.400 系统，而且使用者大多在美国以外的地区。X.400 地址由“属性=值”对组成，相互之间用斜线分开，例如：

```
/C=US/ST=MASSACHUSETTS/L=CAMBRIDGE/PA=360 MEMORIAL DR./CN=KEN SMITH/
```

这个地址指定了一个国家、国内的州、地方城市、个人的地址和通用名字 (Ken Smith)。标准还允许许多其他属性，你可以发送电子邮件给某个人，他的准确电子邮件地址你并不知道，只要你知道关于他的足够其他属性 (例如，公司和职位)。

虽然 X.400 名字使用的方便程度大大低于 DNS 域名，但这个问题对用户代理没有实际意义，因为它们有用户友好的别名 (有时也称为昵称)，允许用户输入或选择一个人的名字，

并得到正确的电子邮件地址。因此，它通常没有必要实际输入这些奇怪的字符串。

最后一点与发送邮件有关的是邮件列表，即允许用户通过一个命令把相同的邮件发送给群发列表中的每个人。如何维护这个邮件列表有两个选择。第一个，可以由用户代理在本地维护。在这种情况下，用户代理只是给每个目标收件人发送一个单独的邮件。

另一个选择是远程维护一个在邮件传输代理上的群发名单。然后，邮件在整个邮件传输系统中被扩散，等同于多个用户给群发列表中的用户发送邮件的作用。例如，如果一群观鸟者有一个称为 `birders` 的邮件列表，该群发列表安装在传输代理 `meadowlark.arizona.edu` 上，那么任何发送到 `birders@meadowlark.arizona.edu` 的邮件都将被路由到美国亚利桑那大学，并在那里被进一步扩散到邮件列表中的所有成员，这些成员可能分布在全世界的任何地方。这个邮件列表中的用户无法分辨这是一个邮件列表，它可能只是 Gabriel O. Birders 教授的个人邮箱。

7.2.3 邮件格式

现在我们从用户接口转到邮件消息格式本身。用户代理发出的邮件必须设置成邮件传输代理能处理的标准格式。首先，我们将考查使用 RFC 5322 的基本 ASCII 电子邮件，然后再考查基本格式的多媒体扩展。RFC 5322 是 Internet 邮件格式的最新版本，最初的 Internet 邮件格式由 RFC 822 描述。

RFC 5322——Internet 邮件格式

邮件由一个基本的信封（作为 SMTP 的一部分由 RFC 5321 描述）、数个头字段、一个空行和邮件体组成。头的每个字段（逻辑上）由一行 ASCII 文本组成，其中包括域名、一个冒号，对于大多数头的字段来说还包括一个值。几十年前完成设计的最初 RFC 822 没有明确区分开信封字段和头字段。虽然 RFC 5322 已经对 RFC 822 做了许多修订，但由于 RFC 822 已经被广泛使用，因此要想完全重新设计是不可能的。在一般的用法中，用户代理创建一个邮件并把它传递给邮件传输代理，然后邮件传输代理利用某些头字段来构造出实际的信封，这个信封有点像老式的邮件与信封混合体。

图 7-10 中列出了与邮件传输相关的主要头字段。“To:” 字段给出了主要收件人的 DNS 地址，邮件系统允许有多个收件人。“Cc:” 字段给出了所有次要收件人的地址。从邮件投递的角度来看，主收件人和次收件人没有区别。这完全是一种心理上的差别，这种差别也许对于相关的人而言很重要，但是对邮件系统来说无关紧要。“Cc:”（Carbon copy，抄送）这个术语有点过时，因为计算机不使用复写纸，但是它已经被约定成俗了。“Bcc:”（Blind carbon copy，密件抄送）字段与 Cc: 字段类似，只是在发送给主收件人和次收件人的所有副本中，这一行被删除了。这个特性允许人们在主收件人和次收件人都不知道的情况下，向第三方发送邮件副本。

接下来的两个字段“From:”和“Sender:”分别指出邮件的撰写者与邮件的发送者，这二者不一定相同。例如，一个公司经理撰写了一个邮件，但她的助理可能是实际发送该邮件的人。在这种情况下，经理被列在“From:”字段中，而她的助理应该出现在“Sender:”字段。“From:”字段是必需的，但是，如果“Sender:”字段与 From: 字段的值相同，那么

可以省略“Sender:”字段。一旦邮件无法投递并且必须被退回发件人时，这些字段就都是必需的了。

邮件头	含义
To:	主要收件人的电子邮件地址
Cc:	次要收件人的电子邮件地址
Bcc:	密件抄送的电子邮件地址
From:	标识了邮件撰写者
Sender:	邮件实际发送者的电子邮件地址
Received:	该行由沿途的每个传输代理填入
Return-Path:	可用于标识邮件发送者的回复路径

图 7-10 RFC 5322 中与邮件传输相关的头字段

在邮件投递的路径上，每个邮件传输代理都会给该邮件加上一行。该行包含“Received:”字段，给出了该代理的标识符、接收到此邮件的日期和时间，以及其他一些可用来查找路由系统中错误的信息。

“Return-Path:”字段由最后一个邮件传输代理添加，主要用来指示如何返回至发件人。理论上，这个信息可以从所有的“Received:”头字段（除了发件人邮箱的名字）收集得到，但实际上很少填充该字段，它一般只包含发件人的地址。

除了图 7-10 中列出的字段以外，RFC 5322 消息还包含了各种供用户代理或者收信人使用的字段。图 7-11 中列出了最常见的一些头字段，它们大部分的含义都不言自明，因此这里我们不对所有的头字段都做详细的介绍。

邮件头	含义
Date:	邮件发出的日期和时间
Reply-To:	回复邮件应采用的电子邮件地址
Message-Id:	稍后引用本邮件的唯一编号
In-Reply-To:	本回复邮件所针对的邮件ID
References:	其他相关邮件的ID
Keywords:	用户选择的关键词
Subject:	一行显示的邮件概要

图 7-11 RFC 5322 邮件头使用的某些字段

有的时候，当邮件的撰写者或者邮件的发送者都不希望看到邮件的回复时，就可以使用“Reply-To:”字段。例如，市场部经理撰写了一个电子邮件消息，向客户介绍一个新产品。秘书发送了这个消息，但“Reply-To:”字段列出的是销售部门的负责人，因为他可以回答问题并处理订单。当发件人有两个电子邮件账号，并希望消息被回复到其中另一个账号时，这个字段也很有用。

“Message-Id:”字段是自动产生的，用于链接邮件（比如，当使用“In-Reply-To:”字段时）并防止重复传递邮件。

RFC 5322 文档明确地指出用户可以发明新的邮件头供自己私人使用，只要这些邮件头（自从 RFC 822 以来）以字符串“X-”开头即可。RFC 822 保证将来的邮件头不会使用以 X-作为开头的名字，以免官方邮件头与私用邮件头之间发生冲突。有时，一些聪明的大学生拼凑出像“X-Fruit-of-the-Day:”或“X-Disease-of-the-Week:”这样的字段，尽管它们不

一定有启发性的含义，但它们却是合法的。

在邮件头之后是邮件体。用户可以在这里放置他们想放的任何内容。有些人用精心设计的签名来结束他们的邮件，这些签名包括对大小某些权威著作的引用、政治声明以及各种各样的声明（例如，XYZ 公司不对我的观点负责；事实上，它甚至不理解我的观点）。

MIME——多用途 Internet 邮件扩展

在 ARPANET 早期，电子邮件只能由文本消息组成，这些消息用英文书写并且以 ASCII 码形式表示。在这样的环境下，RFC 822 完全能够胜任所需的工作：它指定了邮件头，但是把邮件内容完全留给用户自己。在 20 世纪 90 年代，Internet 在全球范围得到广泛使用，发件人希望通过邮件系统发送更为丰富多彩邮件内容的需求越来越强烈，因此原先的这种方法不再够用。主要问题包括了以下情况下的发送和接收两个方面：用带有重音符的语言（例如，法语和德语）来撰写的邮件、用非拉丁字母来撰写的邮件（例如，希伯来语和俄语）、用不带字母的语言来撰写的邮件（例如，中文和日文）以及完全不包含文本的邮件（例如，音频、图像或二进制文档和程序）。

解决方案是多用途 Internet 邮件扩展（MIME，Multipurpose Internet Mail Extensions）。它已被广泛应用于在 Internet 上收发邮件消息，除此之外它还可以描述诸如 Web 浏览器等其他应用的内容。有关 MIME 的信息由 RFC 2045~2047、4288、4289 及 2049 文档描述。

MIME 的基本思想是继续使用 RFC 822 格式（在 RFC 5322 之前 MIME 就已经被提出来了），但在邮件体中增加了结构性，并且为传送非 ASCII 码的邮件定义了编码规则。由于它没有偏离 RFC 822，因此已有的邮件传输代理和协议（基于 RFC 821，现在是 RFC 5321）都可以发送 MIME 消息。需要改变的只是发送和接收程序，而这些可以由用户自己来完成。

MIME 定义了 5 种新的邮件头，如图 7-12 所示。第一个邮件头只是告诉接收邮件的用户代理，它正在处理一条 MIME 消息，以及该消息使用的是哪个版本的 MIME。任何不包含“MIME-Version:”邮件头的消息都被假定是英语明文消息（或者至少只使用 ASCII 字符），并按照这种假定情况下的方式来进行处理。

邮件头	含义
MIME-Version:	标识了MIME版本
Content-Description:	描述邮件的可读字符串
Content-Id:	唯一标识符
Content-Transfer-Encoding:	指出传输时如何打包
Content-Type:	邮件内容的类型和格式

图 7-12 MIME 添加的邮件头

“Content-Description:”头是一个 ASCII 字符串，它指出了邮件包含什么内容。这个头是必需的，这样收件人才能知道是否值得解码和阅读该消息。如果这个字符串说的是“圣巴巴拉仓鼠的照片”，而收到该消息的人并不是一个狂热的仓鼠迷，那么这条消息就可能被丢弃，而不会被解码成一幅高清晰的彩色照片。

“Content-Id:”头标识了邮件的内容，它采用了与标准的“Message-Id:”头相同的格式。

“Content-Transfer-Encoding:”头指出了如何包装邮件体，以便通过网络进行传输。当开发 MIME 时，邮件传送协议（SMTP）只能处理一行不超过 1000 个字符的 ASCII 邮件，

这是一个很关键的问题。ASCII 字符占了每 8 位字节中的 7 位，而二进制数据（比如可执行程序 and 图像）则使用了每个字节的全部 8 位作为扩展的字符集。如果没有任何保障措施，是无法安全传送这种数据的。因此，需要某种方式使得携带二进制数据的邮件看起来就像常规的 ASCII 邮件消息一样。自从 MIME 开发出来之后，SMTP 也做了相应的扩展，允许邮件正文传送 8 位的二进制数据，即使今天二进制数据不进行编码，可能仍然不能被邮件系统正确传送。

MIME 提供了 5 种传送编码方法，还有一个扩充新方案的入口（只是为了以防万一）。最简单的编码方案就是 ASCII 文本消息。每个 ASCII 字符占 7 位，可以被邮件协议直接传送，只要每行都不超过 1000 个字符。

次简单的方案与上一个 ASCII 方案相同，但使用的是 8 位字符。也就是说，从 0 到 255 并且包括 255 在内的所有值都是允许的。使用 8 位编码的邮件仍然必须遵循标准的最大行长度。

然而，存在一些邮件使用了真正的二进制编码。它们是任意的二进制文件，不仅使用全部的 8 位，甚至也不遵循每行 1000 个字符的限制。可执行程序就属于这一类消息。现在，邮件服务器可以协商是否发送二进制（或者不是 8 位的）编码消息，如果双方都不支持该扩展那么就回退到 ASCII 字符。

二进制数据的 ASCII 编码方式称为基于 64 编码（base64 encoding）。在这种方案中，每 24 位编成一个组，每组被分成 4 个 6 位的单元，每个单元被当作一个合法的 ASCII 码字符来发送。编码方式是“A”表示成 0、“B”表示成 1，以此类推。接着是 26 个小写字母、10 个数字，最后是“+”和“/”分别表示成 62 和 63。“=”和“=”分别表示最后一个组只含有 8 位或者 16 位。回车和换行被忽略，所以它们可被任意插入到编码字符流中以便保证每一行足够短。使用这种编码方案后，任意的二进制文本都可以被安全地发送出去，尽管效率不高。在二进制的邮件服务器开发出来之前，这种编码方式非常流行。现在仍然经常有人在用。

对于那些几乎全是 ASCII 字符，只有少量非 ASCII 字符的邮件，base64 编码的效率有些低。发送这种邮件时采用了另一种替代方案，这种方案称为引用可打印编码（quoted-printable encoding）。它正好是 7 位 ASCII 编码，但所有超过 127 的字符都被编码成一个等号后面跟着 2 个用十六进制数字来表示的字符值。控制字符、某些标点符号、数学符号以及结尾空白也用这种方式编码。

最后，如果有足够的理由不使用上述任何一种编码方案，那么还可以通过“Content-Transfer-Encoding:”头说明一种用户自定义的编码。

图 7-12 中显示的最后一个是邮件头实际上是最有趣的一个。它指定了邮件体的本质特性，而且具有超出电子邮件范畴的影响。例如，从 Web 下载的内容被打上 MIME 类型的标签，这样浏览器就知道如何表示该内容。同样地，对于诸如 IP 语音这样的流式媒体和实时传输，作为邮件内容发送时也作同样的处理。

最初，RFC 1521 定义了 7 种 MIME 类型。每种类型都有一个或多个子类型。类型和子类型由一个斜线隔开，比如“Content-Type: video/mpeg”。从此以后，针对每个类型都有数百个子类型被扩充进来。整个过程中，每当开发出某种新的内容类型时要添加额外的表项。分配的类型和子类型列表由 IANA 在线维护 www.iana.org/assignments/media-types。

类型以及相关的常用子类型例子如图 7-13 所示。让我们简单地浏览一下，从“text（文本）”开始。“text/plain（文本/纯文本）”结合起来可用于表达普通邮件，显示的就是收到的，无须编码，也不需要进一步处理。这个选项允许以 MIME 方式传输普通邮件，只需少数几个额外的头。“text/html（文本/网页）”子类型在 Web（RFC 2854）变得流行后允许在 RFC 822 电子邮件中发送网页。“text/xml（文本/可扩展标记语言）”子类型在 RFC 3023 中定义。XML 文档刺激了 Web 的发展。我们将在 7.3 节学习 HTML 和 XML。

类型	子类型实例	描述
text	plain, html, xml, css	不同版本格式的文本内容
image	gif, jpeg, tiff	照片
audio	basic, mpeg, mp4	声音
video	mpeg, mp4, quicktime	影片
model	vrml	3D模型
application	octet-stream, pdf, javascript, zip	由应用程序生成的数据
message	http, rfc822	封装的邮件
multipart	mixed, alternative, parallel, digest	多个类型的组合

图 7-13 MIME 内容类型和子类型实例

下一个 MIME 类型是 image（图像），它可传输静态图像。现在广泛用来存储和传输图像的格式有许多种，它们既可以是压缩的，也可以是未压缩的。其中的一些，包括 GIF、JPEG 和 TIFF 几乎被内置到所有的浏览器中。除此以外还存在很多其他的图像格式和相应的子类型。

audio（音频）和 video（视频）类型分别用于表示声音和运动图像。请注意，video 类型只包含可视信息而不包含声音。如果要传输一部有声电影，那么视频和音频部分必须被分开传输，具体如何操作则要依据所使用的编码系统。第一个定义的视频格式称为运动图像专家组（MPEG, Moving Picture Experts Group），这种格式就是由这组专家设计的，但此后又增加了其他一些格式。除了“audio/basic”以外，在 RFC 3003 中增加了一种新的音频类型“audio/mpeg”，从而使人们可以通过电子邮件发送 MP3 音频文件。“video/mp4”和“audio/mp4”类型说明以新 MPEG 4 格式来存储视频和音频数据。

在其他内容类型之后增加了 model（模型）类型，主要目的是用来描述 3D 模型数据。然而，该类型至今尚未得到广泛的应用。

application（应用）类型是对那些未被其他类型覆盖但又需要应用程序来解释数据的未知格式的统称。例如，我们分别为 PDF 文档、JavaScript 程序和 Zip 压缩包列出了 pdf、javascript 和 zip 子类型。接收到这种邮件内容的用户代理使用第三方库函数或者外部程序来显示内容；显示程序可以集成到用户代理，也可以不和用户代理集成在一起。

通过使用 MIME 类型，用户代理获得了一种处理应用内容的扩展能力。随着新类型应用程序被不断地开发出来，其传输的内容类型也可能是全新的，因此用户代理的这种可扩展处理能力好处很大。但另一方面，许多新形式内容由应用程序执行或者解释也带来了一些危险。显然，运行一个任意可执行程序是有安全隐患的，虽然通过邮件系统传送的可执行程序可能出自“朋友们”。但这种程序可能会对它可以访问到的那部分计算机带来各种令人讨厌的损害，尤其是如果它能够读取和写入文件，以及使用网络时危害更大。不那么明显的是文档格式也可以带来同样的危险。这是因为这些格式（如 PDF）根本是变相的编程

语言。虽然它们被限定在一定范围内解释，但解释器中的错误常常让狡猾的文件挣脱限制的束缚。

因为存在许多应用，除了这些例子外还存在许多各种各样的应用程序子类型。如果没有其他已知的子类型更合适描述邮件，那么作为备用“octet-stream”子类型表示一个无法解释的字节序列。在接到这样一个字节流时，用户代理很可能会显示它，建议用户将其复制到一个文件。然后，在用户大概知道内容是什么样子之后，再决定对该内容做何种后续处理。

最后两个类型对于撰写和处理邮件本身非常有用。**message** 类型使得一个邮件可以被完全封装在另一个邮件中。这种方案对于转发电子邮件很有用。例如，当一个完整的 RFC 822 邮件被封装在另一个外部邮件中时，则应该使用 RFC 822 子类型。类似地，对于封装 HTML 文档也很普遍。**partial** 子类型使得有可能将一个被封装的邮件分成多个部分并分别传输它们（例如，如果被封装的邮件太长）。通过一些参数，接收方能够将各部分按照正确的顺序重新组装起来。

最后一个类型是 **multipart**，它允许一个邮件包含多个部分，并且明确地分出每个部分开始和结束的界限。“**mixed**”子类型允许每个部分类型都不相同，而且无须强制任何额外结构。许多电子邮件程序允许用户在一个文本消息中提供一个或多个附件，这些附件可以 **multipart** 类型发送。

与 **mixed** 相反的是，**alternative** 子类型允许同一个消息被包含多次，但是被表示成两种或多种不同的媒体。例如，一个消息可以按照 3 种形式来发送：纯粹的 ASCII 文本、HTML 和 PDF。一个设计合理的用户代理在接收到这样的消息时，将按照用户的偏好显示。如果可能的话，首选用 PDF 格式显示。第二选择应该是 HTML 格式。如果这两者都不可能，则显示简单的 ASCII 文本。这些部分应该按从简单到复杂的顺序排列，以帮助那些使用非 MIME（pre-MIME）用户代理的收件人也能在一定程度上理解消息的含义（例如，即使非 MIME 的用户也可以阅读简单的 ASCII 文本）。

alternative 子类型也可被用来表示多种语言。在这个上下文意义下，罗塞塔石碑（Rosetta Stone）或许可以被看成是一条早先的 **multipart/alternative** 消息。

有关子类型的其他两个例子是 **parallel** 和 **digest** 子类型。当邮件所有部分必须被同时“观看”时，要使用 **parallel** 子类型。例如，电影往往由一个音频和视频组成。如果这两个频道不是并行播放而是先后播放，那么电影就无法得到应有的播放效果。当多个邮件被打包成一个复合邮件时，要使用 **digest** 子类型。例如，Internet 上的一些讨论组往往收集来自各个本组成员的信息，然后将它们作为单个“**multipart/digest**”邮件定期发送到该讨论组。

下面通过一个例子说明电子邮件如何采用 MIME 类型，图 7-14 给出了一个多媒体邮件。在这里，生日问候信息同时被当作 HTML 和音频文件通过邮件代理传送。假设收件人有播放音频的功能，他的用户代理在展示邮件时将播放声音文件。在这个例子中，声音是作为 **message/external-body** 子类型被引用的，因此用户代理首先必须通过 FTP 获取声音文件 **birthday.snd**。如果收件人没有音频能力，那么歌词就会悄无声息地显示在屏幕上。两部分之间的界限由两个连字符来划分，连字符后面跟着一个字符串（由软件生成的），字符串说明了 **boundary** 参数。

```
From: alice@cs.washington.edu
To: bob@ee.uwa.edu.au
MIME-Version: 1.0
Message-Id: <0704760941.AA00747@cs.washington.edu>
Content-Type: multipart/alternative; boundary=qwertyuiopasdfghjklzxcvbnm
Subject: Earth orbits sun integral number of times

This is the preamble. The user agent ignores it. Have a nice day.

--qwertyuiopasdfghjklzxcvbnm
Content-Type: text/html
<p>Happy birthday to you<br>
Happy birthday to you<br>
Happy birthday dear <b> Bob </b><br>
Happy birthday to you</p>

--qwertyuiopasdfghjklzxcvbnm
Content-Type: message/external-body;
    access-type="anon-ftp";
    site="bicycle.cs.washington.edu";
    directory="pub";
    name="birthday.snd"

content-type: audio/basic
content-transfer-encoding: base64
--qwertyuiopasdfghjklzxcvbnm--
```

图 7-14 由 HTML 和音频多个部分组成的邮件

请注意，在这个例子中，“Content-Type”邮件头出现在 3 个位置上。在顶层，它指出该邮件由多个部分组成。在每个部分中，它指出了该部分的类型和子类型。最后，在第二个部分的正文中，它必须告诉用户代理应该获取什么类型的外部文件。为了显示这种用法的细微不同之处，我们在这里使用了小写字母，尽管所有的邮件头是不区分大小写的。同样地，对于不是按 7 位 ASCII 编码的外部邮件体，则需要提供“Content-Transfer-Encoding”头。

7.2.4 邮件传送

既然我们已经描述了用户代理和邮件消息，现在已经做好一切准备来进一步考查邮件传输代理如何将邮件从发件人中继给收件人。邮件传送采用的协议是 SMTP。

移动邮件最简单的方法是建立一个从源机器到目标机器的传输连接，然后在该连接上传输邮件。这是 SMTP 的最初工作方式。然而，经过多年的发展，邮件的传输逐步区分出了两种使用 SMTP 的不同方式。第一种使用方式是邮件提交（mail submission），如图 7-7 所示的电子邮件体系结构中的第 1 步。这一步表示用户代理把邮件提交给邮件系统。第二种使用方式是邮件传输代理之间的邮件传送（图 7-7 的第 2 步）。这个序列将邮件从发送邮件传输代理传递到接收邮件传输代理，全程只有一跳。完成最终邮件的交付使用了不同的协议，我们将在下一节中描述。

在本小节，我们将描述基本的 SMTP 协议和它的扩展机制。然后，我们将讨论如何利

用它完成邮件提交和邮件传送。

SMTP（简单邮件传输协议）及其扩展

在 Internet 上, 发送电子邮件的计算机首先与目标计算机的 25 号端口建立一个 TCP 连接, 然后在此连接上传送电子邮件。在这个端口上监听的是邮件服务器, 它遵守简单邮件传输协议 (SMTP, Simple Mail Transfer Protocol)。这个服务器接受入境连接请求、执行某些安全检查, 并接受传递过来的邮件。如果一个邮件无法被投递, 则向邮件发送方返回一个错误报告, 该错误报告包含了无法投递邮件的第一部分。

SMTP 是一个简单的 ASCII 协议。这不是一个弱点, 而是一种特性。因为使用 ASCII 文本, 使得协议更加易于开发、测试和调试。通过手动发送命令就可以进行测试, 而且记录的消息易于阅读。现在大多数应用程序级的 Internet 协议都是以这种方式工作的 (比如 HTTP)。

我们将考查负责传递消息的邮件服务器如何完成一个简单邮件的传输过程。在建立了与 25 端口的 TCP 连接后, 作为客户端的发送机器等待接收机器首先说话, 这个接收机器是作为服务器运行的。服务器开始发送一个文本行给客户端, 该行表明了自己的标识, 并告诉客户机是否已经准备好接收邮件。如果服务器没有这样做, 那么客户端就释放连接, 稍后再次尝试与服务器联系。

如果服务器愿意接收电子邮件, 则客户端声明这封电子邮件来自于谁以及将要交给谁。如果接收方确实存在这样的收件人, 则服务器指示客户发送邮件; 然后客户发送邮件, 服务器予以确认。因为 TCP 提供了可靠的字节流传输, 所以这里不需要校验和。如果还有更多的电子邮件需要传输, 那么现在可以继续发送。当两个方向上所有的电子邮件都交换完毕后, 连接被释放。图 7-14 所示的发送邮件会话实例如图 7-15 所示, 图中还包含了 SMTP 所使用的数字代码。客户端发送的行用 “C:” 标识, 而服务器发送的行则用 “S:” 标识。

来自客户的第一条命令实际意味着 HELO。在 HELLO 的各种 4 个字符缩写组合中, 这种缩写相比于其竞争者而言有诸多优点。随着时间的流逝, 为什么所有的命令必须是 4 个字符的原因现在已经无从追究了。

在图 7-15 中, 邮件只发送给一个收件人, 因此这里只使用了一条 RCPT 命令。RCPT 命令可以将一个邮件发送给多个收件人。每个邮件被单独确认或拒绝。即使发给有些收件人的邮件遭到了拒绝 (由于接收方并不存在这样的收件人), 邮件也可以被发送给其他的收件人。

最后, 尽管客户端命令的语法被严格地限定为四字符, 但是回复消息的语法就不那么严格了。实际上, 只有数字代码才具有真正的意义。每一种实现都可以在数字代码后面加上它所喜欢的任何字符串。

基本 SMTP 运作良好, 但它在几个方面存在不足。首先它不包括认证。这意味着例子中的 FROM 命令可以为所欲为地给出任何发件人的地址。这个特性对于发送垃圾邮件相当有用。其次, SMTP 传输的是 ASCII 消息而不是二进制数据。这就是为什么需要 Base64 MIME 内容传送编码方案。然而, 使用该编码的邮件在传输时带宽使用效率低下, 这对传输大邮件是个问题。最后, SMTP 发送的邮件以明文形式出现。它没有任何加密功能可用来说提供防止窥探隐私的措施。

```

S: 220 ee.uwa.edu.au SMTP service ready
C: HELO abcd.com
S: 250 cs.washington.edu says hello to ee.uwa.edu.au
C: MAIL FROM: <alice@cs.washington.edu>
S: 250 sender ok
C: RCPT TO: <bob@ee.uwa.edu.au>
S: 250 recipient ok
C: DATA
S: 354 Send mail; end with "." on a line by itself
C: From: alice@cs.washington.edu
C: To: bob@ee.uwa.edu.au
C: MIME-Version: 1.0
C: Message-Id: <0704760941.AA00747@ee.uwa.edu.au>
C: Content-Type: multipart/alternative; boundary=qwertyuiopasdfghjklzxcvbnm
C: Subject: Earth orbits sun integral number of times
C:
C: This is the preamble. The user agent ignores it. Have a nice day.
C:
C: --qwertyuiopasdfghjklzxcvbnm
C: Content-Type: text/html
C:
C: <p>Happy birthday to you
C: Happy birthday to you
C: Happy birthday dear <bold> Bob </bold>
C: Happy birthday to you
C:
C: --qwertyuiopasdfghjklzxcvbnm
C: Content-Type: message/external-body;
C:   access-type="anon-ftp";
C:   site="bicycle.cs.washington.edu";
C:   directory="pub";
C:   name="birthday.snd"
C:
C: content-type: audio/basic
C: content-transfer-encoding: base64
C: --qwertyuiopasdfghjklzxcvbnm
C: .
S: 250 message accepted
C: QUIT
S: 221 ee.uwa.edu.au closing connection

```

图 7-15 alice@cs.washington.edu 给 bob@ee.uwa.edu.au 发送一个邮件

为了处理这些以及其他与邮件处理相关的问题，SMTP 已经被修订过，增加了一个扩展机制。这个机制是 RFC 5321 标准的强制性执行部分。带有扩展功能的 SMTP 就称为扩展的 SMTP（ESMTP, Extended SMTP）。

想要使用扩展版本的客户端必须发送 EHLO 消息，而不是原先的 HELO。如果该消息遭到了服务器的拒绝，那么说明对方是一个普通的 SMTP 服务器，客户端应该以从前的方式与其交互。如果 EHLO 消息被服务器接受，服务器就用它支持的扩展给予回复。然后客户端可以使用这些扩展功能中的任何一种。几种常见的扩展功能如图 7-16 所示。该图给出了扩展机制所用的关键字，以及这些关键字具有的新功能说明。我们不会对这些扩展做详

细的讨论。

关键字	描述
AUTH	客户认证
BINARYMIME	服务器接受二进制邮件
CHUNKING	服务器接受巨型邮件
SIZE	在发送前检查邮件大小
STARTTLS	切换至安全传输 (TLS, 参见第8章)
UTF8SMTP	国际化地址

图 7-16 某些 SMTP 扩展功能

为了更好地理解 SMTP 协议和本章描述的其他协议是如何工作的,最好试用这些协议。无论使用什么协议,首先必须来到一台已接入 Internet 的计算机前面。在 UNIX (或 Linux) 系统上,在 shell 中键入:

```
telnet mail.isp.com 25
```

请用你 ISP 的邮件服务器的 DNS 名字来替换 mail.isp.com。在 Window XP 系统中,首先单击“开始”(Start),然后单击“运行”(Run),并且在对话框中键入上述命令。在 Vista 或 Windows 7 机器上,你必须首先安装 telnet 程序 (或等价的其他程序),然后你自己启动该程序运行。这条命令将与那台机器的 25 端口建立一个 telnet (也就是 TCP) 连接。25 号端口是 SMTP 端口;参考图 6-34,它列出了部分常用的端口。你可能会得到类似这样的回应:

```
Trying 192.30.200.66...
Connected to mail.isp.com
Escape character is '^]'.
220 mail.isp.com Smail #74 ready at Thu, 25 Sept 2002 13:26 +0200
```

前三行来自于 telnet 程序,告诉你它在做什么。最后一行来自远程机器上的 SMTP 服务器,声明它愿意与你通话,并愿意接受电子邮件。为了找出该 SMTP 服务器可以接受哪些命令,请键入:

```
HELP
```

如果服务器愿意接受你的邮件,那么从现在开始,你有可能看到像图 7-16 所示的命令序列。

邮件提交

最初用户代理和负责发送的邮件传输代理运行在同一台计算机上。在这样的设置中,发送邮件所需要做的只是通过我们刚才所描述的对话框,用户代理与本地邮件服务器交互完成邮件的发送。但是,这种设置现在已经不常用了。

用户代理通常运行在笔记本电脑、家用电脑和移动电话上。它们并不总是能连接到 Internet。邮件传输代理则运行在 ISP 和公司的服务器上,它们始终和 Internet 保持连接。这种差异意味着,在波士顿的用户代理可能需要联系他在西雅图的常规邮件服务器发送邮件,因为这个用户正在旅行中。

就其本身而言，这种远程通信不会构成任何问题。它恰好是 TCP / IP 协议旨在支持的功能。然而，ISP 或公司通常不希望任何远程用户将邮件提交给它的邮件服务器再传递到其他地方。ISP 或公司并没有为了提供公共服务而运行邮件服务器。此外，这种开放邮件中继（open mail relay）会吸引垃圾邮件发送者，因为这种方式提供了一种清洗原始发件人的途径，从而使得邮件更难以被识别为垃圾邮件。

鉴于这些因素，SMTP 通常被用在提交电子邮件时启用了 AUTH 扩展。这个扩展可以让服务器检查客户端的凭据（用户名和密码），以便确认自己是否应该为该客户提供邮件服务。

在 SMTP 提交邮件的方式上还存在几个其他方面的差异。例如，587 端口优先于端口 25，SMTP 服务器可以检查和纠正由用户代理发送的邮件格式。有关更多使用 SMTP 来提交邮件受到的限制，请参阅 RFC 4409。

邮件传送

一旦接收到来自用户代理发送的邮件，邮件传输代理便使用 SMTP 将该邮件传送给接收邮件传输代理。要做到这一点，发送者必须使用目标地址。考虑图 7-15 中发给 bob@ee.uwa.edu.au 的邮件。该邮件应该被传递给哪个邮件服务器？

为了确定要联系的正确邮件服务器，必须咨询 DNS。在上一节中，我们介绍了 DNS 如何包含多种类型的资源记录，包括 MX 记录，或邮件交换器。在这种情况下，发出一次 DNS 查询，请求 ee.uwa.edu.au 域的 MX 记录。这个查询得到的响应是一个包含一个或多个邮件服务器名字和 IP 地址的有序列表。

然后，发送方的邮件传输代理与邮件服务器 IP 地址的端口 25 建立一个 TCP 连接，通过该服务器能到达接收方的邮件传输代理，并使用 SMTP 来中继消息；然后接收方的邮件传输代理将发给用户 Bob 的邮件放置在其正确的邮箱里，供 Bob 在以后的时间读取。如果接收方有一个庞大的电子邮件基础设施，那么这种本地交付邮件的步骤可能涉及在计算机之间移动邮件。

在传递邮件的过程中，从最初的邮件传输代理到最终的邮件传输代理一跳完成邮件的传输，在这传输过程中不涉及任何中间服务器。然而，邮件传递过程则可能发生需要多次传递的情况。我们已经描述过的一个例子就属于这种情况：当邮件传输代理实现了一个邮件列表时。在这种情况下，邮件要被传递到列表中的每个收件人。邮件在维护该列表的邮件传输代理上被复制成多个邮件，分别发给列表中每个成员的地址。

作为邮件中继的另一个例子，我们来看即使 Bob 从麻省理工学院毕业后，还可以通过地址 bob@alum.mit.edu 与他联系。与通过多个账户阅读邮件方式不同的是，Bob 可以安排把发送到上述邮件地址的邮件转发到 bob@ee.uwa.edu。在这种情况下，发送到 bob@alum.mit.edu 的邮件将经历两次交付过程。首先，它被发送到邮件服务器 alum.mit.edu；然后，它又被转发到邮件服务器 ee.uwa.edu.au。这一系列的每一步都是一个完整并且独立的交付过程，只关注到邮件传输代理。

时下另一个需要考虑的是垃圾邮件。今天发送的 10 个消息中有 9 个是垃圾邮件（McAfee, 2010）。虽然很少有人愿意接收更多的垃圾邮件，但难以避免，因为垃圾邮件通常伪装成普通邮件。在接受邮件之前，进行额外的安全检查能减少接收垃圾邮件的机会。

给 Bob 的邮件发自 `alice@cs.washington.edu`。接收端的邮件传输代理可以在 DNS 查找发送端的邮件传输代理,即检查 TCP 连接另一端的 IP 地址是否与 DNS 域名相匹配。更一般地,接收代理可以在 DNS 中查询发送域,看它是否有一个邮件发送政策。这种信息往往由 TXT 和 SPF 记录提供,它还能表明可以进行其他检查。例如,从 `cs.washington.edu` 发出的邮件永远来自主机 `june.cs.washington.edu`,如果发送方的邮件传输代理不是 `june`,那么邮件肯定有问题。

如果其中任何一项检查失败,那么邮件可能被伪造成用了一个假的发送地址。在这种情况下,应该丢弃该邮件。然而,通过检查的邮件也并不意味着就一定不是垃圾。检查只能确保该邮件似乎来自网络中它声称的区域。我们的想法是应该强制垃圾邮件发送者在发送垃圾邮件时必须使用其正确的发送地址。这样可以使得垃圾邮件在不受欢迎时更容易被识别和删除。

7.2.5 最后传递

我们的邮件几乎要被交付了。它已经抵达 Bob 邮箱。剩下的工作就是将邮件的一个副本传送到 Bob 的用户代理以便显示。这是图 7-7 体系结构中的第 3 步。这个任务很简单,在早期 Internet 时代,当用户代理和邮件传输代理作为不同的进程运行在同一台机器上时就已经完成了。邮件传输代理简单地把新邮件写到邮箱文件的结尾处,然后用户代理只需检查邮箱文件看是否添加了新邮件。

如今,运行在 PC、笔记本电脑或移动电话上的用户代理可能与运行 ISP 或公司邮件服务器的设备不是同一台机器。用户希望无论在哪里都能够远程访问他们的邮件。他们既想在工作中访问邮件,也想回家时从家用电脑上网读取邮件,甚至出差时用笔记本电脑或者度假时从所谓的网吧来访问电子邮件。他们还希望能够脱机工作,需要时重新连接 Internet 以便接收入境邮件和发送出境邮件。此外,每个用户或许要运行几个用户代理,具体使用哪个代理则取决于当时什么电脑最方便。甚至有可能在同一时间运行多个用户代理。

在这样的设置中,用户代理的工作是给出邮箱内容的一个概览,并允许用户远程对邮箱进行操纵。提供这种功能的协议有许多种,但 SMTP 不是其中之一。SMTP 是一种“基于推”(push-based)的协议,它获取一个邮件,并且连接到远程服务器来传递邮件。邮件的最终交付不能以这种方式实现,因为两个原因:第一,邮件传输代理上的邮箱必须可以连续存储;第二,用户代理可能在 SMTP 试图中继邮件的那一刻无法连接到 Internet 上。

IMAP——Internet 邮件访问协议

最终交付邮件使用的主要协议之一是 Internet 邮件访问协议 (IMAP, Internet Message Access Protocol)。RFC 3501 定义了 IMAP 协议版本 4。为了使用 IMAP,邮件服务器必须运行 IMAP 服务器,它负责监听端口 143。用户代理必须运行 IMAP 客户端。客户端与服务器连接,并开始发出如图 7-17 中列出的命令。

首先,如果要启用安全机制(为了加密邮件和命令)客户端必须启动一个安全传输,然后登录或以其他方式向服务器验证自己。一旦成功登录后,可以执行很多命令,包括列表显示文件夹和邮件、获取邮件或者甚至部分邮件、给邮件标记供以后删除,并将邮件组

命令	描述
CAPABILITY	列表显示服务器能力
STARTTLS	启用安全传输(TLS, 参见第8章)
LOGIN	登录服务器
AUTHENTICATE	用其他方式登录服务器
SELECT	选择一个文件夹
EXAMINE	选择一个只读文件夹
CREATE	创建一个文件夹
DELETE	删除一个文件夹
RENAME	重命名一个文件夹
SUBSCRIBE	在活动集中添加文件夹
UNSUBSCRIBE	从活动集中删除文件夹
LIST	列出可用的文件夹
LSUB	列出活动的文件夹
STATUS	获取一个文件夹的状态
APPEND	往文件夹内添加一个邮件
CHECK	获取一个文件夹的断点
FETCH	从一个文件夹内获取邮件
SEARCH	在一个文件夹内搜索邮件
STORE	变更邮件标志
COPY	复制一个文件夹内的某个邮件
EXPUNGE	删除做了删除标志的邮件
UID	用唯一标识法发命令
NOOP	什么也不做
CLOSE	删除邮件并关闭文件夹
LOGOUT	登出并关闭连接

图 7-17 IMAP（版本 4）命令

织到各文件夹内。为了避免混淆，请注意，这里我们使用了术语“文件夹”（folder），以便和本节的其余部分保持一致，即一个用户有一个由多个文件夹组成的邮箱。然而，在 IMAP 规范中采用了术语“邮箱”（mailbox）来代替文件夹。因此，一个用户有许多 IMAP 邮箱，每个邮箱通常表现为提交给用户的一个文件夹。

IMAP 还有许多其他功能。它能不通过邮件号而是使用属性来寻址邮件（例如，Alice 发给我的第一个邮件）。还可以在服务器上执行搜索功能，找到满足某个特定标准的邮件，因此客户只需要获取这些邮件。

IMAP 是较早使用的最终交付协议——**邮局协议版本 3**（POP3, Post Office Protocol, version 3）的改进版。POP 3 由 RFC 1939 说明，它是个非常简单的协议，只支持较少的功能，而且其典型使用方式不太安全。它通常把邮件下载到用户代理所在的计算机上，而不是留在邮件服务器上。虽然这种处理方式使得服务器的工作更加容易，但用户难以为生。用户希望多台不同的计算机上阅读邮件非常困难，而且如果用户代理所在的计算机发生故障，那么所有的电子邮件将可能永久地丢失。尽管如此，你仍然会发现有人还在使用 POP 3。

因为邮件服务器和用户代理之间运行的协议可以由同一家公司提供，因此也可以使用其他专有协议。Microsoft Exchange 就是一个运行其专有协议的邮件系统。

Webmail

一种日益流行可提供电子邮件服务的是 Web，它可以利用其接口来发送和接收邮件，这种方式有望替代 IMAP 和 SMTP。目前被广泛使用的 Webmail 系统包括谷歌 Gmail、微软 Hotmail 和雅虎 Mail。Webmail 是一个软件实例（在这种情况下，就是邮件用户代理），它利用 Web 为用户提供邮件服务。

在这样的体系结构中，为了接收来自端口 25 并且使用 SMTP 的用户邮件，服务提供商照常运行邮件服务器。然而，此时的用户代理与以前是不同的。它不再是作为一个独立的程序运行，而是通过网页提供了一个用户界面。这意味着用户可以使用自己喜欢的任何浏览器来访问他们的邮件，以及发送新邮件。

我们至今还没有学习 Web，但现在先给出一个关于它的简短介绍，或许以后你会回过头再看它。当用户进入服务提供商的电子邮件网页时，会看到一个表单，要求用户输入登录名和密码。然后登录名和密码被发送到服务器，在服务器端进行验证。如果登录成功，那么服务器会找出用户的邮箱，并建立一个网页，该网页列出了邮箱的大致内容；然后将该网页发送到用户的浏览器显示。

页面上显示的许多邮箱表项都是可点击的，因此邮件可以被读取、删除等。为了使界面用户对用户的行为有所反应，网页往往包括 JavaScript 程序。这些程序运行在本地客户端以便响应任何一个本地事件（比如鼠标点击）、在后台下载或上传邮件，或者准备下一个邮件的显示或提交一个新邮件。在这种模型中，邮件提交采用了普通的 Web 协议，把数据张贴到 URL。Web 服务器负责将邮件注入到传统的邮件传递系统中，这个我们已经在前面描述过了。出于安全方面的考虑，可使用标准的 Web 协议。这些协议本身涉及加密网页，而不管网页的内容是否是一个邮件消息或是一般网页内容。

7.3 万 维 网

Web 是万维网（World Wide Web）的俗称，它是一个体系结构框架。该框架把分布在整个 Internet 数百万台机器上的内容链接起来供人们访问。Web 刚出现时在瑞士被科研人员用来相互之间协同设计高能物理实验，仅十年间它就演变成今天被数百万人认为的“Internet”应用。它的迅速普及和流行源自这样的事实：它易于初学者使用并且提供了丰富多彩的图形界面。通过这些界面用户可以访问巨大的信息财富，内容几乎覆盖了每一个可以想到的主题，从 aardvarks（土豚）到 Zulus（祖鲁族人）什么都有。

Web 诞生于 1989 年的欧洲原子能研究中心 CERN。最初的想法是帮助大型研究组成员通过修改报告、计划、绘图、照片和其他文档的方式来进行合作，这些文档由粒子物理实验产生，并且研究组的成员通常分散在好几个国家或好几个时区。将文档链接成 Web 的提议由 CERN 物理学家 Tim Berners-Lee 提出，18 个月后第一个（基于文本的）原型系统投入运行。该系统的公开文档发表在 Hypertext'91 会议，它立即引起了另一个研究组的注意，该研究组由伊利诺伊大学的 Marc Andreessen 领导，他最终开发了第一个图形浏览器。这就是 Mosaic 浏览器，正式发布于 1993.2。

正如他们所说的，其余的现在已经成为历史。Mosaic 是如此的受欢迎，一年后

Andreessen 离开学校组建了网景通信公司（Netscape Communications Corp.），公司目标是开发 Web 软件。接下来的 3 年，网景的 Netscape Navigator 和微软的 IE 浏览器进入了一场浏览器大战，每一个都试图捕捉这个新兴市场的更大份额，为此疯狂地加入比对手更多的功能（而且导致了更多的错误）。

从 20 世纪 90 年代到本世纪初，网站和网页（称为 Web 内容）成指数倍地增长，直到达到具有数百万计网站和数十亿网页的规模。这些网站中的一小部分盛极一时，网站和它们背后的公司主要定义了 Web，正如今天人们所体验的那样。这些公司包括书店（亚马逊于 1994 年成立，市值 500 亿美元）、跳蚤市场（易趣，1995 成立，市值 300 亿美元）、搜索（谷歌，1998 成立，市值 150 亿美元）和社会网络（Facebook，2004 成立，私人公司，价值超过 150 亿美元）。到 2000 年期间，许多 Web 公司一夜之间晋升数百万美元身价，当最后表明原来一切都只是炒作时，几乎接近破产。这一切甚至还有一个特殊名称，即所谓的点 com 时代（dot com era）。新的想法仍然丰富着 Web 世界。许多新想法来自年轻的学生。例如，当 Mark Zuckerberg 开始创建 Facebook 时是哈佛大学的学生；Sergey Brin 和 Larry Page 创建 Google 时是斯坦福大学的学生。也许你会想出下一件什么大事来。

1994 年，CERN 和 MIT 签署了建立万维网联盟（W3C，World Wide Web Consortium）的协议。W3C 是一个组织，它致力于进一步开发 Web、对协议进行标准化，并鼓励站点之间实行互操作。Berners-Lee 担任了联盟的主管。从那时起，已经有几百所大学和公司加入了该联盟。尽管现在关于 Web 的书籍数不胜数，但获取关于 Web 最新信息的最佳之处（很自然地）还是在 Web 本身。W3C 联盟的主页是 www.w3.org，感兴趣的读者可以从那里找到涵盖该联盟所有文档和活动的页面链接。

7.3.1 体系结构概述

站在用户的角度看，Web 由大量分布在全球范围的内容组成，这些内容以 **Web 页面**（Web page）或简称为**页面**（page）的形式表示。每个页面可以包含指向其他页面的**链接**（link），这些页面可以分布在全球任何地方。用户单击一个链接就可以跟随这个链接来到它所指向的页面。这个过程可无限地重复下去。让一个页面指向另一个页面的想法现在称为**超文本**（hypertext），这种想法在 1945 年由一个卓有远见的 MIT 电子工程系教授 Vannevar Bush 发明（Bush，1945），也就是说早在 Internet 被发明出来之前就已经有了 Web。事实上，它在商业计算机之前就已经存在了，虽然几所大学生生产出来的粗糙原型机能填满大型会议室，而且能力还不及一个现代化的袖珍计算器。

通常观看页面的程序称为**浏览器**（browser）。Firefox、Internet Explorer 和 Chrome 是比较流行的浏览器。浏览器取回所请求的页面，对页面内容进行解释，并在屏幕上以恰当的格式显示出来。页面内容本身可能是文本、图像和格式化的命令混合体，表现的形式多种多样：可以表现成传统的文档形式，或者表现成其他内容的形式（比如视频），或者是一个能产生图形界面的程序，用户通过该界面实行与网页的交互方式。

页面的照片如图 7-18 左边一般所示。这是华盛顿大学计算机科学与工程系的一个页面。这个页面显示了文本和图形元素（大多数内容太小无法阅读）。页面中的某些部分与指向其他页面的链接有关。与另一个页面相关的一小段文字、一个图标、一个图像等都称为超链

接 (hyperlink)。为了跟随一个链接, 用户将鼠标光标放在页面区域的链接部分 (这会使光标发生变化), 然后单击链接。点击链接只是告诉浏览器去获取另一个页面的简单方式。在 Web 初期, 链接通过下划线和彩色文本来突出强调, 以便使它们脱颖而出。如今, Web 页面的创作者已经有各种方法来控制链接区域的外表, 因此一个链接可能会作为一个图标出现, 或当鼠标滑过它时改变外观。正是页面的创造者使得链接在视觉上表现鲜明, 从而提供了一个可用的接口。

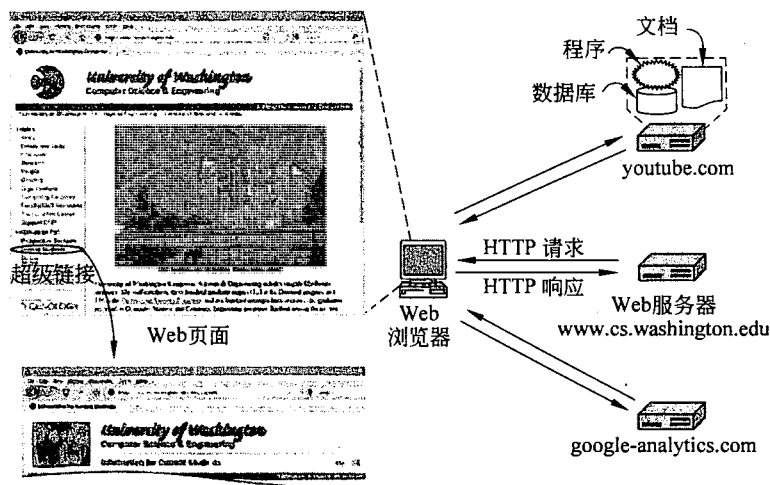


图 7-18 Web 的体系结构

系里的学生跟随一个链接到一个特别为他们而设计内容的页面, 能学到更多东西。通过点击圈起来的区域就可以访问该链接。然后, 浏览器抓取新的一页并显示出来, 如图左下角所示的那部分。除了这个例子外, 第一页还包含着指向数十个其他网页的链接。每个其他页面可以由包括在同一台机器上的页面组成 (就像第一页一样), 或者由位于世界各地机器上的内容组成, 对此用户无从知晓。页面的抓取由浏览器完成, 无须任何来自用户的帮助。因此, 在观看网页的内容时, 网页内容在机器之间的移动是无缝进行的。

页面显示背后的基本工作模型如图 7-18 所示。在这里, 客户机器上的浏览器正在显示一个 Web 页面。每一页的抓取都是通过发送一个请求到一个或多个服务器, 服务器以页面的内容作为响应。抓取网页所用的“请求-响应”协议是一个简单的基于文本协议, 它运行在 TCP 之上, 就像 SMTP 一样。这个协议就是所谓的超文本传输协议 (HTTP, HyperText Transfer Protocol)。内容可能只是一个磁盘读取的文档, 或者是数据库查询和程序执行的结果。如果每次显示的是相同的一个文档, 则称该网页为静态页面 (static page)。相反, 如果每次显示的是程序按需产生的内容, 或者页面本身包含了一个程序, 则称该网页为动态页面 (dynamic page)。

一个动态的页面每次显示时本身表现可能是不同的。例如, 电子商店的首页可能对每个访问者显示的内容不尽相同。如果一个书店的顾客在过去买了一些推理小说, 那么当这位顾客访问商店的主页后, 可能会看到突出显示的新的惊悚小说, 而另一位更喜欢烹饪的顾客可能首先映入眼帘的是新的烹饪书籍。网站如何跟踪哪位顾客喜欢什么我们马上就会说明。简单地说, 其答案涉及一种 Cookie。

在图中，浏览器接触三个服务器获取了两个页面，这三个服务器分别是 edu、cs.washington、youtube.com 和 google-analytics.com。来自这些不同服务器的内容集成在一起通过浏览器显示。显示细化了网页处理的范围，主要取决于什么样的内容。除了渲染文字和图形外，它可能还涉及播放一段视频，或者运行一个脚本把作为页面一部分的用户界面呈现出来。在这种情况下，cs.washington.edu 服务器提供了主网页，youtube.com 服务器提供了一段嵌入的视频，而 google-analytics.com 服务器没有提供任何用户可见的内容，但它追踪访问网站的用户。我们稍后将详细讨论跟踪器。

客户端

现在让我们来看图 7-18 中 Web 浏览器这边的详细情况。基本上，一个浏览器是一个程序，它可以显示 Web 页面并且捕获鼠标在显示页面上点击的表项。当一个表项被击中时，浏览器就跟踪相应的超链并获取被选中的页面。

当 Web 最初被建立时，为了让一个页面指向另一个 Web 页面，很显然需要某些机制来命名和定位页面。尤其是，在显示一个被选中的页面之前，首先必须回答 3 个问题：

- (1) 这个页面叫什么？
- (2) 这个页面在哪里？
- (3) 如何访问这个页面？

如果每个页面都以某种方式被分配了一个唯一的名字，那么在标识页面时就不会存在任何歧义。但是，问题还没有解决。考虑把人与页面作个对比。在美国，几乎每个人都有一个人社会保险号，因为任何两个人都不可能拥有相同的社会保险号，因此社会保险号是一个具有唯一性的标识符。然而，如果你仅仅知道一个社会保险号，那么你是无法找到该保险号所有者的地址，当然你也无法知道到底应该用英语、西班牙语还是汉语来给他写信。Web 基本上也有同样的问题。

Web 选择的解决方案是用一种能同时解决上述 3 个问题的方式来标识页面。每个页面被分配一个统一资源定位符 (URL, Uniform Resource Locator)，用来有效地充当该页面在全球范围内的名字。URL 包括 3 个部分：协议（也称为方案 (scheme)）、页面所在机器的 DNS 名字，以及唯一指向特定页面的路径（通常是读取的一个文件或者运行在机器上的一个程序）。一般情况下，路径是一个模仿文件目录结构的层次名字。然而，如何解释路径是服务器的事，而且路径可能反映了实际的目录结构，也可能不反映实际的目录结构。

作为一个例子，图 7-18 显示的页面的 URL 是：

```
http://www.cs.washington.edu/index.html
```

这个 URL 由 3 部分组成：协议 (http)、主机的 DNS 域名 (www.cs.washington.edu) 和路径名 (index.html)。

当用户点击一个超链，浏览器就执行一系列的步骤来获取该超链指向的网页。让我们跟踪点击例子中链接时所发生的步骤：

- (1) 浏览器确定 URL（通过观察选中的什么）。
- (2) 浏览器请求 DNS 查询服务器 www.cs.washington.edu 的 IP 地址。
- (3) DNS 返回 128.208.3.88。

(4) 浏览器与 128.208.3.88 机器的 80 端口建立一个 TCP 连接, 80 端口是 HTTP 协议的知名端口。

(5) 浏览器发送 HTTP 报文, 请求/index.html 页面。

(6) www.cs.washington.edu 服务器发回页面作为 HTTP 响应, 例如发送文件/index.html。

(7) 如果该页面包括需要显示的 URL, 那么浏览器经过同样的处理过程获取其他 URL。在这种情况下, URL 包括多个取自 www.cs.washington.edu 的内嵌图像、一个取自 youtube.com 的内嵌视频和一个取自 google-analytics.com 的脚本。

(8) 浏览器显示页面/index.html, 如图 7-18 所示。

(9) 如果短期内没有向同一个服务器发出其他请求, 那么释放 TCP 连接。

许多浏览器会在屏幕底部的状态栏中显示它们目前正在执行哪一步。这样, 当性能较差时, 用户可以看到是由于 DNS 没有响应、服务器没有响应, 或者干脆正在一个缓慢或拥塞的网络上传输页面。

URL 设计是开放式的, 在某种意义上它很简单, 允许浏览器使用多种协议去获得各种不同的资源。事实上, 已经定义了针对各种其他协议的 URL。图 7-19 列出了稍微简化了的常见形式。

名字	用途	实例
http	超文本(HTML)	http://www.ee.uwa.edu/~rob/
https	安全的超文本	https://www.bank.com/accounts/
ftp	FTP	ftp://ftp.cs.vu.nl/pub/minix/README
file	本地文件	file:///usr/suzanne/prog.c
mailto	发送邮件	mailto:JohnUser@acm.org
rtsp	流式媒体	rtsp://youtube.com/montypython.mpg
sip	多媒体呼叫	sip:eve@adversary.com
about	浏览器信息	about:plugins

图 7-19 某些公共的 URL 方案

让我们简单地浏览上述列表。http 协议是 Web 的母语, 即 Web 服务器讲的语言。HTTP 代表超文本传输协议 (HyperText Transfer Protocol)。我们将在本节后面详细讨论它。

ftp 协议用于通过 FTP 访问文件, 这是 Internet 的文件传输协议。FTP 早在 Web 之前就有, 已经被使用超过 30 年。通过提供一个简单的可点击界面而不是一个命令行界面, Web 使得用户更加易于获得存放在世界各地众多 FTP 服务器上的文件。这种获取信息的方式改进是蔚为壮观 Web 增长的原因之一。

使用 file 协议或者更简单地只要给出一个文件名就有可能通过 Web 页面来访问本地文件。这种方法并不需要一个服务器。当然, 它仅适用于本地文件, 而不能用于远程文件。

mailto 协议并没有真正抓取网页, 但反正是有用的。它允许用户从 Web 浏览器发送电子邮件。大多数浏览器针对一个 mailto 链接会以启动用户的邮件代理作为回应, 返回一个已填写好地址字段的邮件。

rtsp 和 sip 协议用于创建流式媒体会话和音视频的呼叫。

最后, about 协议是一种惯常方式, 主要用来提供有关浏览器的信息。例如, about: plugins 链接会导致大部分浏览器显示一个网页, 该网页列出它们利用浏览器扩展可以处理的

MIME 类型，这种浏览器扩展称为插件（plugin）。

总之，URL 的设计不仅允许用户浏览网页，而且允许用户运行一些旧的协议，比如 FTP 和电子邮件，以及涉及音频和视频的新协议；除此之外，还提供了访问本地文件和浏览器信息的便利方法。这种方法不需要专门为其他服务设计特殊的用户界面程序，而是把几乎所有的 Internet 访问集成到单一的程序：Web 浏览器。如果这种想法不是由一个在瑞士研究实验室工作的英国物理学家发明的，那么它可能很容易得到一些软件公司广告的梦想计划的支持。

抛开所有这些漂亮的属性，越来越多的 Web 使用已经暴露出 URL 方案中的固有弱点。一个 URL 指向一个特定的主机，但有时引用一个页面而无须告诉它在哪里也是很有用的。例如，对于被大量引用的页面，需要有多个相距甚远的副本，以便减少网络流量。但用户没有办法说：“我希望得到页面 xyz，但我不关心在哪里得到它。”

为了解决这类问题，URL 已经被推广到统一资源标识符（URI，Uniform Resource Identifiers）。某些 URI 告诉浏览器如何定位资源，这些就是 URL；其他 URI 告诉了资源名字，但没有说明到哪里可以找得到它，这些 URI 就称为统一资源名（URN，Uniform Resource Names）。如何书写 URI 的规则由 RFC 3986 给出说明，而不同 URI 方案的使用则由 IANA 负责跟踪。除了在图 7-19 列出的方案外，还有许多不同类型的 URI，但这些图 7-19 列出的这些方案在今天 Web 的使用中占据了主导地位。

MIME 类型

为了能够显示新页面（或任何页面），浏览器必须了解其格式。为了让所有的浏览器都了解所有网页，网页必须以一种标准化的语言编写，这个语言就称为 HTML。它是 Web 的通用语（现在）。我们将在本章后面详细讨论该语言。

虽然浏览器基本上是一个 HTML 解释器，但大多数的浏览器有众多的按钮和功能，帮助用户更容易地浏览网页。大多数浏览器有一个返回到前一页、前进到下一页的按钮（只有当用户操作了回退动作后），和一个直接通往用户首选页的按钮。大多数浏览器还有一个按钮或菜单项用来在给定页面上设置书签，而且还提供了另一个显示书签列表的按钮或菜单项，这种方式下只需要点击几下鼠标就有可能重新审视书签。

正如我们的例子所显示的那样，HTML 页面包含了丰富的内容元素，而不只是简单的文本和超文本。为了增加一般性，并不是所有的页面都需要包含 HTML。一个页面可能由一段 MPEG 格式的视频、一个 PDF 格式的文件、一张 JPEG 格式的照片、一首 MP3 格式的歌曲，或任何数百种其他文件类型的内容组成。由于标准的 HTML 页面可以链接到任何一种格式的内容，因此当浏览器命中一个它不知道如何解释的页面时，问题就产生了。

不是把浏览器越做越大，使其内置的解释器能适应快速增长的文件类型，相反大多数浏览器都选择了一个更为一般性的解决方案。当一台服务器返回一个页面时，它同时也返回了一些关于此页面的其他信息。这些信息包括页面的 MIME 类型（见图 7-13）。具有 text / html 类型的页面可被直接显示，就像其他一些内置类型的网页一样。如果 MIME 类型不是一个内置的类型，那么浏览器查阅其 MIME 类型表，以便确定如何显示该页面。此表将 MIME 类型与观众关联。

有两种可能的方式：插件和辅助应用程序。插件（plug-in）是一个第三方代码模块，

作为扩展被安装到浏览器中,如图 7-20(a)所示。常用的插件例子有 PDF、Flash 和 Quicktime,主要用来呈现文档和播放音视频。由于插件运行在浏览器内部,因此它们可以访问当前的页面,也可以修改页面的外观。

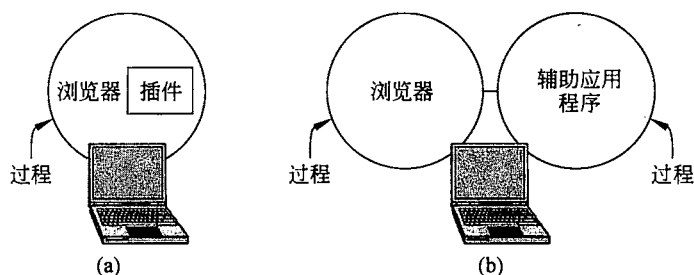


图 7-20

(a) 浏览器插件; (b) 辅助应用程序

每种浏览器都有一组过程,所有的插件必须实现这些过程,这样浏览器才可以调用插件。例如,通常浏览器的基本代码会调用一个专门过程,将待显示的数据传递给插件。这组过程就是插件的接口,它与特定的浏览器相关。

另外,浏览器也为插件提供了一组它自己的过程,以便向插件提供服务。浏览器接口较为典型的过程包括申请和释放内存、在浏览器的状态栏上显示一条消息,以及向浏览器查询有关的参数。

在一个插件被使用以前,首先必须安装该插件。一般的安装过程是,用户到插件的 Web 站点下载一个安装文件。执行安装文件将插件解压缩,并且执行适当的调用以便注册该插件的 MIME 类型,并将浏览器与该插件关联起来。浏览器通常预载了一些流行的插件。

另一种扩展浏览器的方式是使用一个辅助应用程序(helper application)。这是一个作为独立进程运行的完整程序,如图 7-20(b)所示。因为辅助应用程序是独立运行的程序,因此接口与浏览器非常靠近。它通常只是接受一个存储了待显示内容的临时文件名字,然后打开该文件并显示其内容。典型地,辅助应用程序是一些独立于浏览器而存在的大型程序,比如 Microsoft Word 或者 PowerPoint。

许多辅助应用程序使用 MIME 的 application 类型。因此,目前已经定义了相当多的子类型,例如, application/vnd.ms-powerpoint 表示 PowerPoint 文件。Vnd 代表特定于开发者的格式。通过这种方式,URL 就可以直接指向一个 PowerPoint 文件,并且当用户单击它时,浏览器自动启动 PowerPoint,并将待显示的内容传递给它。辅助应用程序并不受限于只能使用 MIME 的 application 类型。例如, image/x-photoshop 代表 Adobe Photoshop。

因此,可以将浏览器配置成能处理几乎无限多种文档的类型,而且无须改变浏览器本身。现代 Web 服务器常常被配置成具有数百种“类型/子类型”的组合,每当安装一个新程序时,新的类型/子类型组合也随之被加入进来。

针对一些子类型,比如 video/mpeg,当多个插件和辅助应用程序都适用的时候,就会产生冲突。当注册的最后一个覆盖掉已有与 MIME 类型关联的插件或者辅助应用程序,从而为自己捕获类型时,会发生什么呢?因此,安装一个新的程序可能会改变浏览器处理现有类型的方式。

浏览器也能够打开本地的文件,而不一定非得从远程的 Web 服务器上取回文件,就像

没有网络一样。然而，浏览器需要某种方式来确定文件的 MIME 类型。标准的方式是操作系统将文件的扩展与 MIME 类型关联起来。在典型的配置中，当浏览器打开 `foo.pdf` 时，它就会用 `application/pdf` 插件，而当打开 `bar.doc` 文件时则通过 `application/msword` 辅助应用程序调用 Word。

同样地，由于许多程序都希望（实际上是渴望）处理某些类型的文件，比如说 `mpg`，所以这里还可能发生冲突。在安装过程中，供经验丰富用户使用的程序常常会针对它们处理的 MIME 类型和扩展名，给用户显示一些复选框，并且允许用户选择合适的程序，这样就不会无意中覆盖掉已有的关联关系。面向消费者市场的程序则假设用户根本不知道什么是 MIME 类型，它们只是尽可能地抓取到所有的类型，而根本不关心以前安装的程序做过什么。

能够将浏览器扩展成拥有处理大量新类型的能力确实非常方便，但这也导致了麻烦。当 Windows 个人计算机上的浏览器抓取了一个扩展名为 `exe` 的文件时，它知道这个文件是可执行程序，因而没有对应的辅助应用程序。很显然它应该运行这个程序，然而，这可能是一个巨大的安全漏洞。一个恶意的 Web 站点所需做的全部事情只是生成一个包含许多图片（比如电影明星或体育明星）的 Web 页面，并且将所有的图片都链接到一个病毒上。于是，单击任何一幅图片都会导致取回一个未知的、可能恶意的可执行程序，并且在用户的机器上运行。为了预防这样的不速之客入侵，Firefox 和其他浏览器配置成特别小心地自动运行未知程序，但是，并不是所有的用户都懂得什么样的配置才是安全的而不是便利的。

服务器端

关于客户端的介绍已经很多了，现在让我们来看看服务器端。正如我们在前面所看到的，当用户键入一个 URL 或者单击一行超文本时，浏览器会解析 URL，并且将 `http://` 和下一个斜线之间的那部分解释成待查找的 DNS 名字。有了服务器的 IP 地址以后，浏览器与该服务器的端口 80 建立一个 TCP 连接；然后它发送一条命令，其中包含了 URL 的剩余部分，即该服务器上某个页面的路径；最后服务器返回该页面供浏览器显示。

乍一看，Web 服务器与图 6-6 中的服务器非常类似。该图中的服务器得到一个待查找的文件名字，并且通过网络返回结果。在这两种情况下，服务器在它的主循环中执行如下步骤：

- (1) 接受来自客户端（浏览器）的 TCP 连接。
- (2) 获取页面的路径，即被请求文件的名字。
- (3) 获取文件（从磁盘上）。
- (4) 将文件内容发送给客户。
- (5) 释放该 TCP 连接。

现代的 Web 服务器具有更多的功能，但本质上这就是 Web 服务器在最简单情况下所做的工作，即获取一个包含网页内容的文件。对于动态内容，第三步必须替换成运行一个程序（由路径确定），该程序能返回网页的内容。

然而，为了单位时间处理多个请求，Web 服务器的实现具有不同的设计。简单设计的一个问题是文件访问通常成为瓶颈。相对程序执行而言，读磁盘的速度很慢，而且通过操作系统调用重复读取相同的磁盘文件。另一个问题是一次只能处理一个请求。文件或许会